

2025年度
修士論文

SN開発におけるデバッグシステムの開発

指導教員	アサノ デービッド	教授
	不破 泰	肩書き

2025年1月30日提出

信州大学大学院 総合理工学研究科 工学専攻 情報数理・融合システム分野
24W6047A
辻村篤志

あらまし

近年、無線センサネットワーク（WSN）や IoT は、環境モニタリングやスマートシティなど多様な分野での活用が進み、実機上で通信プロトコルを検証する重要性が高まっている。先行研究では、UML 状態遷移図から通信プロトコルのコードを自動生成し、無線通信デバイス上で動作検証できる環境が構築されていた。しかし、デバッグ方法はエディタのコンソールへのログ出力に依存しており、通信処理が高速かつ複雑に進行する場合に状態遷移や内部変数の推移を逐次追跡することが困難であった。

本研究では、実機で実行されるプロトコルの内部動作を直感的に把握できるデバッグ支援システムを提案し、ブラウザ上で動作する可視化デバッグ環境として実装した。提案システムは、状態遷移ログと変数更新ログを統一形式で出力し、ログ解析により状態列と各ステップに対応する変数値スナップショット列を再構成する。これにより、状態遷移図のハイライト表示、変数ウォッチ表示、ステップ実行による逐次確認を、同一のステップ番号に基づいて連動させる。加えて、設計に用いる状態遷移図をデバッグシステム上に表現することで、通信状態の遷移をより視覚的にとらえることを可能にした。また、同期モードでは、2 台のデバイスから同時に受信したログに時刻情報を付与し、イベント列を統合して同期タイムラインを生成することで、同一時刻における 2 台のノードの状態と変数を比較可能とした。

評価では、実行ログを用いて単一ノードの追跡性向上と、同期タイムラインによる複数ノードの相互作用把握を確認し、提案システムが実機検証におけるデバッグ作業の効率化に有効であることを示した。

目次

第1章	はじめに	1
1.1	研究背景	1
1.2	当研究室における先行研究	1
1.3	先行研究の課題	2
1.4	本研究の目的	2
第2章	先行研究の提案システム	3
2.1	システムの概要	3
2.2	実現事項	4
2.3	課題	4
第3章	提案システムの概要と設計方針	5
3.1	設計方針	5
3.2	システムの構成	6
3.3	各コンポーネントの概要	7
3.3.1	コード生成コンポーネント	8
3.3.2	組込みデバイスコンポーネント	8
3.3.3	デバッグ支援システムコンポーネント	9
第4章	デバッグ支援システムの設計・実装	10
4.1	デバッグ支援システムの概要	10
4.2	実装技術と採用理由	10
4.2.1	フロントエンドフレームワーク	10
4.2.2	UI フレームワーク	11
4.2.3	状態遷移図の描画ライブラリ	11

4.2.4	ブラウザ上でのシリアル受信	11
4.3	動作モードとシステム構成	11
4.3.1	動作モードの分類	11
4.3.2	モード間で共通に用いる設計要素	12
4.4	ファイル解析モードの設計・実装	13
4.4.1	ファイル解析モードの概要	13
4.4.2	ファイル解析モードの UI 設計	13
4.5	シリアル受信モードの設計・実装	14
4.5.1	シリアル受信モードの概要	14
4.5.2	シリアル受信モードの UI 設計	15
4.5.3	ブラウザ上でのシリアルモニタ実装	17
4.5.4	同期タイムラインの生成	17
4.6	ログ設計	22
4.6.1	ログ生成の設計	22
4.7	ログ解析の設計（共通処理）	23
4.7.1	概要（解析の目的と処理の流れ）	23
4.7.2	入力ログ形式とログ行の分類	24
4.7.3	解析結果として構築するデータ構造	25
4.7.4	変数値スナップショットの再構成方針（継承と更新）	25
4.7.5	解析結果例	26
4.8	ステップ番号に基づく共通表示ロジック	26
4.8.1	共通の前提：ステップ番号 i と状態スナップショット	26
4.8.2	ステップ実行のロジック	27
4.8.3	変数ウォッチ機能のロジック	27
4.8.4	Mermaid による状態遷移図描画	27
第5章	評価	31
5.0.1	評価の目的	31
5.0.2	評価対象	31
5.0.3	評価で用いるプロトコル	32
5.1	ファイル解析モードにおけるデバッグの評価	32

5.1.1	評価シナリオ	32
5.1.2	ログ入力と解析結果の生成 (states/vars)	32
5.1.3	状態遷移ログ表示	36
5.1.4	変数ウォッチ表示 (状態ステップに対応した変数値追跡)	37
5.1.5	状態遷移図表示 (JSON 入力→ Mermaid 変換→描画)	38
5.1.6	ステップ操作・スライド操作 (表示要素の同期更新)	40
5.2	シリアル受信モード固有機能の評価 (2 ノード同期タイムライン)	41
5.2.1	評価シナリオ	41
5.2.2	シリアルモニタによるログ受信	42
5.2.3	同期タイムライン生成と表示	43
5.2.4	同期解析の有効性 (定性的観察)	45
5.3	評価のまとめ	46
第6章	考察	48
6.1	既存手法とのギャップと、本方式が埋めた点	48
6.1.1	「時間順の羅列」から「状態遷移という意味単位」へ	48
6.1.2	「ログ行の追跡」から「同時更新による照合」へ	48
6.1.3	「単体観察」から「同一時刻の並列比較」へ	49
6.2	システムの限界・制約	49
6.2.1	同期方式の適用範囲	49
6.2.2	ログ形式への依存	50
6.2.3	変数追跡対象の限定	50
6.2.4	評価範囲および拡張性に関する制約	50
6.3	今後の課題	51
6.3.1	遠距離環境での同期解析への対応	51
6.3.2	ログ形式依存の低減 (パースの強化)	51
6.3.3	AI を用いたログパースの補助	51
6.3.4	変数追跡の運用負担の軽減	51
6.3.5	効果の定量評価	52
6.3.6	多ノード化・大規模ログへの対応	52
6.4	本研究のまとめ	52

目 次

2.1	先行研究システムの概要図	3
3.1	従来のデバッグ方法（コンソール出力）	6
3.2	提案システムの概要図	7
3.3	どこでモデム（左）と GPS モジュール（右）	9
4.1	モード選択のフローチャート	12
4.2	ファイル解析モードのレイアウト	14
4.3	シリアルモニタのレイアウト	15
4.4	シリアルポート選択ドロップダウン	16
4.5	同期モードのレイアウト	17
4.6	同期解析におけるタイムライン（構築前）	19
4.7	構築された同期タイムラインの例	19
4.8	ログ解析の処理フロー	24
4.9	変数値スナップショットの継承と更新	26
4.10	Mermaid によって描画された状態遷移図の例	29
5.1	状態遷移ログ表示（左：10 行表示、右：30 行表示）	36
5.2	変数ウォッチ表示（変数選択）	37
5.3	変数ウォッチ表示（ステップによる変数の変化）	38
5.4	Mermaid による状態遷移図の描画例	40
5.5	ステップ実行の様子	41
5.6	評価に用いるデバイス構成	42
5.7	シリアルモニタによるログ受信の様子（左：中継器、右：端末）	43
5.8	同期化された状態遷移表示例	45

5.9 同期デバッグ実行例（タイムライン選択に連動した複数ノードの同時表示）	46
--	----

表 目 次

第 1 章

はじめに

1.1 研究背景

近年、無線センサネットワーク（Wireless Sensor Network：WSN）や IoT（Internet of Things）は、環境モニタリングやスマートシティ、インフラ監視、農業、ヘルスケアなど、さまざまな分野での応用が期待されている。実際、IoT 全体の接続デバイス数は急速に増加している。IoT Analytics によれば、2023 年時点で約 166 億台、2024 年には約 188 億台に達し、2030 年には約 400 億台に到達すると予測されている [1]。また、WSN の普及も市場規模の面から拡大を続けており、Grand View Research の報告では、産業用 WSN（Industrial WSN）の市場は 2023 年に約 51.9 億ドルと推定され、2024 年から 2030 年にかけて年平均 12.1%の成長が見込まれている [2]。

しかし、これらのシステムを構築するためには、複雑な通信プロトコルの設計やデータ処理の設計および実装、動作検証が求められ、開発者には幅広い専門知識と多大な工数が必要となる。さらに、無線通信環境の構築やマイコン設定など導入時のハードルも高いため、これらの研究や開発はシミュレーション環境やエミュレータ上での検証にとどまることが多い。しかし、シミュレーション環境では実機特有の挙動や外部環境の影響を完全に再現することが難しく、実機での検証が不可欠である場合も多いため、実機でのプロトコル検証を容易にするための支援環境の整備が求められている。

1.2 当研究室における先行研究

当研究室の先行研究では、UML の一種である**状態遷移図**上で無線通信プロトコルの動

作を定義し、そこから自動生成したプログラムを汎用ハード上で動作させ、プロトコルを実機で動作検証できる環境が構築されていた（旭ら [3]、小林ら [4]）。

1.3 先行研究の課題

このシステムにより簡易的に通信プロトコルの動作検証が可能となったが、デバッグ方法はエディタ上のコンソールへのテキストログ出力に依存しており、通信処理が高速かつ複雑に進むため逐次的な状態遷移を追跡することが難しかった。その結果、複雑な動作を含むプロトコルに対しては、異常動作の原因究明や、通信動作を把握しづらく、開発効率や教育的利用の観点では十分でない点が課題として残されていた。

1.4 本研究の目的

本研究の目的は、先行研究で課題となっていたデバッグ効率の低さを改善し、実機でのプロトコル検証をより効率的かつ視覚的に行える環境を実現することである。そのために、無線通信デバイス上で実行された動作をログとして出力し、それを可視化・ステップ実行できる仕組みを開発した。これにより、プロトコルの動作を逐次的に把握でき、異常動作の原因究明や通信動作の把握を効率化するとともに、教育やチーム開発、応用実証に適した支援環境を提供することを目指す。

第 2 章

先行研究の提案システム

2.1 システムの概要

先行研究で提案されたシステムの概要図を図 2.1 に示す。先行研究では、無線通信プロトコルの設計・実装・動作検証を支援するシステムが提案されてきた。このシステムは主に次の二つのコンポーネントから構成されている。第一に、UML の状態遷移図を用いてプロトコルの動作を定義し、そこからファームウェア（実機上で動作するプログラム）を自動生成するコンポーネントである。第二に、生成されたファームウェアを実行するための環境（ハードウェア）を提供するコンポーネントである。これにより、状態遷移図で定義されたプロトコルが実機上で動作し、その動作を検証できる環境が提供されてきた。つまりユーザは状態遷移図さえ記述すれば、無線通信プロトコルの実装および動作検証を容易に行うことができる。

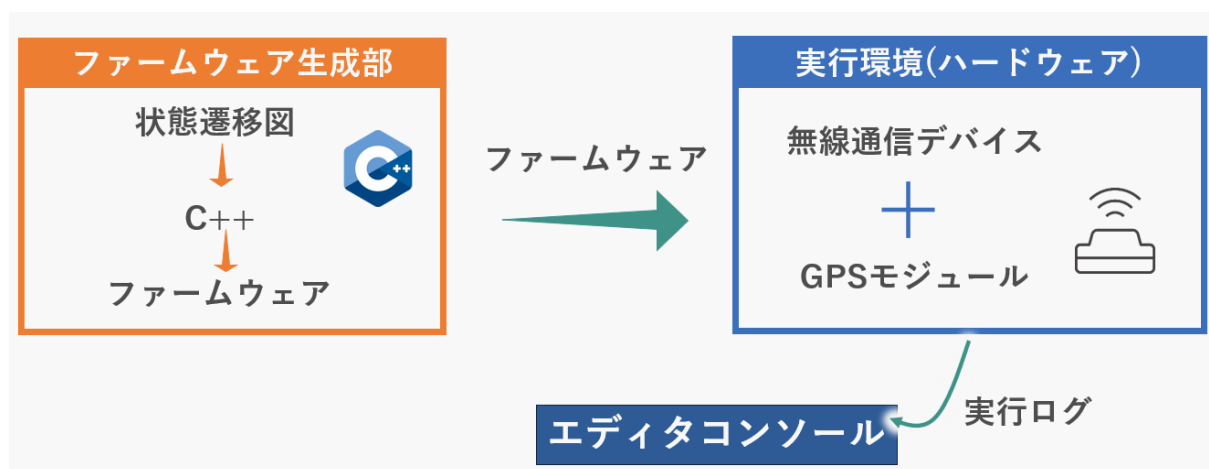


図 2.1: 先行研究システムの概要図

2.2 実現事項

先行研究システムでは、主に以下の機能が実現されている。

- 状態遷移図からのコード自動生成：UML の状態遷移図を用いてプロトコルの動作を定義し、そこから C++ プログラムを自動生成する機能が提供されている。これにより、プロトコル設計者は視覚的に動作を定義でき、手動でのコーディング作業を削減できる。
- 実機上での動作検証環境：生成されたコードは、無線モデム搭載のマイコンを組み合わせたハードウェア上で実行される。これにより、シミュレーション環境では捉えきれない実機特有の挙動を検証できる。

2.3 課題

先行研究システムには以下の課題が存在する。

- 状態遷移図の記法の制約：先行研究における状態遷移図からのコード自動生成は、特定の記法に依存しており、複雑なプロトコル動作を表現する際に制約がある。これにより、設計者が意図する動作を完全に反映できない場合がある。
- デバッグ効率の低さ：デバッグ方法がコンソールへのテキストログ出力に依存しており、通信処理が高速・複雑に進むため逐次的な内部状態（状態、変数値など）を追跡することが難しい。これにより、複雑な動作を含むプロトコルに対しては、異常動作の原因を把握しづらい。

これらの課題を解決するために、本研究ではデバッグ効率の向上に焦点を当て、実機でのプロトコル検証をより効率的に行える環境を提案する。なお、状態遷移図からのコード生成部については、[5] で改善が図られているため、そちらを参照されたい。

第 3 章

提案システムの概要と設計方針

本研究では、先行研究システムにおいて課題となっていたデバッグ効率の低さを改善することを目的として、新たなデバッグ支援システムを提案する。本章では、提案システムの概要と設計方針について述べる。まず、先行研究における課題を整理し、次に設計方針と満たすべき要求を示す。最後に、システム全体の構成と各コンポーネントの役割を説明する。

3.1 設計方針

先行研究システムでは、状態遷移図を用いて通信プロトコルを設計し、実機上で動作検証を行うことが可能であった。一方で、デバッグ方法は主に図 3.1 のようにコンソールへのログ出力に依存しており、状態遷移の流れや内部変数の変化を逐次的に把握することが難しいという課題があった。通信処理が高速に進行する場合、ログが短時間に大量出力されるため、必要な状態や変数の変化点を追跡しにくく、原因箇所の特定に手作業の負担が生じる。

本研究ではこの課題を踏まえ、実機上で実行される通信プロトコルの内部動作を、**状態遷移**に基づいて段階的に確認できるデバッグ支援環境の構築を設計方針とした。具体的には、プロトコルの実行状況をログとして取得し、それを解析して可視化することで、動作の流れをステップ単位で追跡できる仕組みを目指す。

設計上の要求を以下にまとめる。

- プロトコルは状態遷移に基づいて動作すること。

```
STATE:IDLE
current_slot = 0
STATE:RECEIVE_SLOT_INFO
STATE:CHECK_RECEIVED_PACKET
STATE:WAIT_OWN_SLOT
current_slot = 1
STATE:WAIT_OWN_SLOT
STATE:WAIT_OWN_SLOT
current_slot = 2
STATE:WAIT_RANDOM_DELAY
STATE:CREATE_PACKET
STATE:TRANSMIT
.
.
.
```

図 3.1: 従来のデバッグ方法（コンソール出力）

- ログは状態遷移ログと変数更新ログを解析対象とし、解析可能な統一された形式で出力されること。
- 状態遷移をステップ単位で追跡できること。
- 状態遷移図、ログ、変数値が同一ステップに基づいて同期して提示されること。
- 先行研究の枠組みと親和性を保ち、研究室内で継続利用および拡張が容易であること。
- 単一ノードの解析に加えて、複数ノードの比較にも拡張可能であること。

3.2 システムの構成

システム全体の構成を図 3.2 に示す。提案システムは、先行研究システムに対してデバッグ支援機能を付加する形で構成されている。全体としては、ファームウェアを生成す

るファームウェア生成部、通信プロトコルを実行する実行環境（組み込みデバイス）と、その動作を外部から確認するためのデバッグ支援システムからなる。

組み込みデバイスでは、状態遷移に基づいて通信プロトコルが実行され、その実行過程に関する情報がログとしてシリアル経由で出力される。デバッグ支援システム側では、出力されたログを収集して解析し、状態遷移や変数値の推移を可視化する。これにより、従来は把握が困難であった動作過程を、ステップ単位で確認可能とする。また、従来はPlatformIO上のシリアルモニタでログを確認していたが、本システムではブラウザ上で動作するシリアルモニタをデバッグ支援システムに実装し、ログの受信と解析を一元的に行えるようにした。

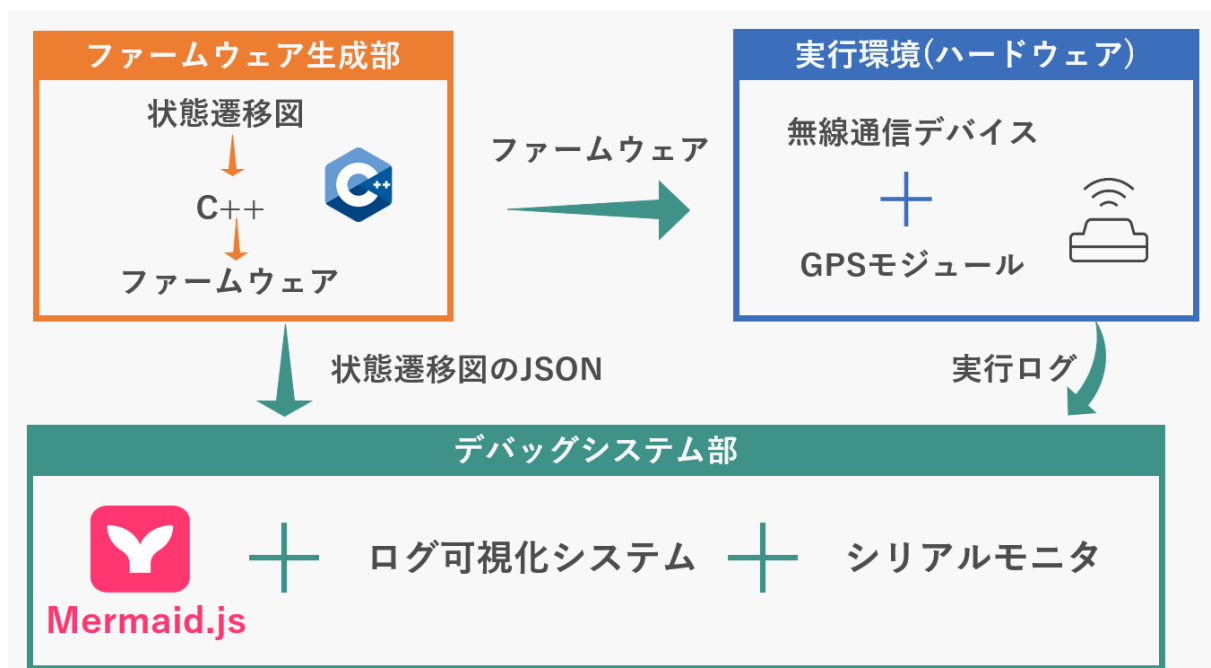


図 3.2: 提案システムの概要図

3.3 各コンポーネントの概要

提案システムは、主に以下のコンポーネントから構成されている。

- 状態遷移図からコードを自動生成するコンポーネント。
- 通信プロトコルを実行する組み込みデバイスコンポーネント。
- ログ収集と可視化を担うデバッグ支援システムコンポーネント。

なお、コード生成およびデバッグコンポーネントはPC上で動作し、組み込みデバイスは外部デバイスであることを留意されたい。

3.3.1 コード生成コンポーネント

コード生成コンポーネントは、UMLの状態遷移図を解析し、通信プロトコルの動作を表現するプログラムおよびファームウェアを自動生成する役割を担う。

本コンポーネントでは状態遷移図の構造を解析し、状態、遷移、イベントおよびアクションを抽出する。抽出した情報に基づき、通信プロトコルの動作を表現するC++コードを生成し、状態遷移はswitch-case文を用いて実装される。生成されたコードはPlatformIOを用いてビルドされ、ファームウェアが生成される。このファームウェアは次節で述べる組み込みデバイスに書き込まれた後、その上で実行される。

なお、本研究の主眼はコード生成手法そのものではなく、生成されたプログラムの実行ログに対するデバッグ支援環境の構築にあるため、コード生成手法の詳細については文献[5]を参照されたい。

3.3.2 組み込みデバイスコンポーネント

本システムにおけるプロトコルの実行環境である組み込みデバイスを図3.3に示す。本研究では、SAMD21マイコンと無線通信モジュールを一体化した無線通信デバイス（株式会社サーキットデザイン製どこでモデム）を用いる。

本デバイスは429MHz帯で動作する汎用的な無線通信モジュールであり、技術基準適合証明（技適）を取得しているため、実際に電波を用いた通信プロトコルの検証が可能である。また、429MHz帯は中山間地域において回折性能に優れており、低消費電力で長距離通信が可能なLPWA（Low Power Wide Area）通信に適している。この特性により、登山者見守りシステムなどへの応用も期待できる。さらに、本デバイスはマイコンと通信モジュールが一体化しているため、外部配線や追加ハードウェアを必要とせず、開発や実験環境の構築を容易に行える。また、本デバイスはPCとUSBを用いたシリアル通信が可能であり、デバッグ支援システムと連携してログを送信できる。

必要に応じてGPSモジュールを併用することで、位置情報の取得や1PPS信号を用いた時刻同期により、高精度同期を前提とするTDMA方式などにも対応可能となる。な



図 3.3: どこでモデム（左）と GPS モジュール（右）

お、GPS モジュールは使用しない場合には取り外すこともできる。また、その他のセンサを用いた組み込みシステムを構築する場合には、別途センサモジュールを接続して利用することも可能である。

3.3.3 デバッグ支援システムコンポーネント

デバッグ支援システムは、本研究の中核であり、組込みデバイスから出力される実行ログを収集し、解析結果を可視化して提示する役割を担う。従来はコンソール上に流れるログを人手で追跡する必要があり、状態遷移の流れや変数値の変化点を把握しにくいという課題があった。本システムでは、状態遷移ログと変数更新ログを解析対象として統一形式で扱い、ログを状態遷移の出現順に基づくステップ単位へ再構成することで、状態遷移図、状態ログ、変数値表示を同一ステップに基づいて同期させる。

さらに、ログの入力経路として、実行ログファイルアップロードに加えてブラウザ上で動作するシリアルモニタを備える。これにより、組込みデバイスの実行ログを受信し、そのまま同一画面内で解析と可視化へ移行できるため、ログ受信とデバッグ作業を一元化できる。また、複数ポートからのログを同時に取り扱うことで、複数ノード間の挙動比較に発展させることも可能である。

以上により、本コンポーネントは、実機上で進行する通信プロトコルの動作を段階的に確認し、原因箇所の特定や相互作用の理解を支援する。本コンポーネントの具体的な設計と実装については、次章で詳述する。

第 4 章

デバッグ支援システムの設計・実装

4.1 デバッグ支援システムの概要

本研究で提案するデバッグ支援システムは、ブラウザ上で動作するフロントエンドアプリケーションとして実装されている。本システムは、組込みデバイスから得られる実行ログと状態遷移図情報を入力として、(1) 状態遷移の進行状況の可視化、(2) 変数値の追跡、(3) ステップ実行による逐次確認、(4) 複数ノード間の同期デバッグ、を支援することを目的とする。

本章では、まず実装技術と採用理由を述べ、次に動作モードを「ファイル解析モード」と「シリアル受信モード」に整理する。その後、モード間で共通に用いる設計要素として、ログ設計、ログ解析、ステップ番号に基づく共通表示ロジック、変数ウォッチ、状態遷移図描画とハイライト表示を順に説明する。

4.2 実装技術と採用理由

4.2.1 フロントエンドフレームワーク

UI およびロジックの構築には React.js（以下、React）を採用した。React はコンポーネント指向のフレームワークであり、画面要素を状態（state）と結び付けて管理できる。本研究のデバッグ支援システムでは、ステップ操作に伴い、状態遷移図のハイライト表示や変数値の表示が頻繁に更新される。このため、状態変化に応じた UI 更新を宣言的に

記述できる React は適している。さらに、表示部品をコンポーネントとして分割できるため、可読性・保守性の向上にも寄与する。

4.2.2 UI フレームワーク

UI のスタイリングおよびレイアウト設計には Bootstrap を使用した。Bootstrap は、あらかじめ定義された CSS クラスおよび UI コンポーネントを提供し、レスポンシブな画面設計を容易に実現できる。本システムでは、多数の情報（状態遷移図、ログ、変数値など）を同一画面上に整理して表示する必要があるため、グリッドシステムやカードコンポーネントを活用し、視認性と操作性の両立を図った。

4.2.3 状態遷移図の描画ライブラリ

状態遷移図の描画には Mermaid.js を用いる。Mermaid は図をテキスト（構文）として記述し、その文字列から図を生成できるため、状態遷移図 JSON から構文を生成する設計とすることで、自動描画とステップ連動のハイライト表示を実現できる。

4.2.4 ブラウザ上でのシリアル受信

シリアル受信モードでは Web Serial API を用い、ブラウザから直接シリアルポートに接続してログをリアルタイムに受信する。従来使用していた別アプリケーション（platformIO 等）のシリアルモニタを使用せず、ログの受信、解析、可視化を同一 UI 内で一元化できる点を重視した。

4.3 動作モードとシステム構成

4.3.1 動作モードの分類

本システムは入力経路に基づき、ファイル解析モードとシリアル受信モードの2つの動作モードを備える。図 4.1 に使用するモードの選択フローチャートを示す。

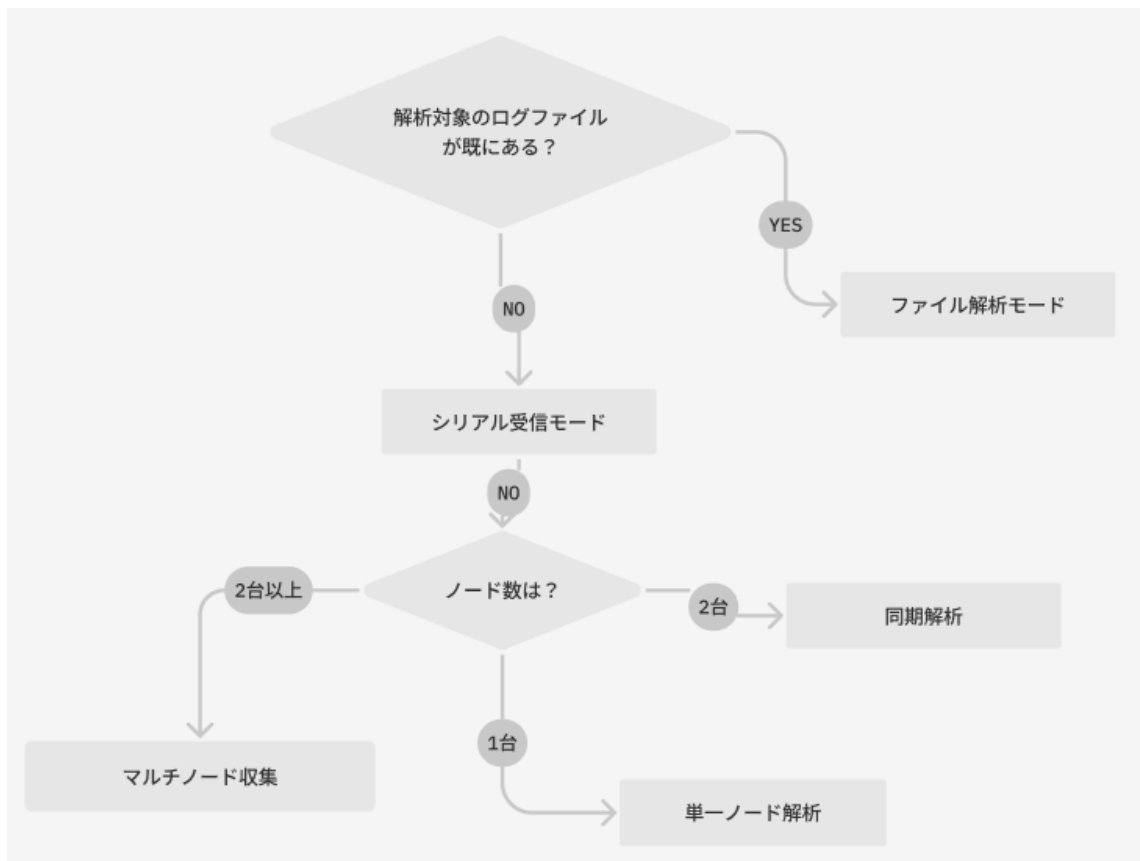


図 4.1: モード選択のフローチャート

ファイル解析モードは、既を取得済みのログファイルを入力として解析と可視化を行う。シリアル受信モードは、組込みデバイスからログを受信し、目的に応じて解析と可視化、または収集を行う。

シリアル受信モードはノード数と目的に応じて、単一ノード解析、同期解析（2台）、マルチノード収集（2台以上）の3形態に分かれる。単一ノード解析と同期解析は受信ログを解析して可視化デバッグを行う。マルチノード収集は、主としてログを受信してコピーを行うことを目的とし、解析は対象外とする。なお以上の3形態はユーザーが指定するものではなく、接続されているノード数と目的に応じて自動的に切り替わる。

4.3.2 モード間で共通に用いる設計要素

両モードはログの入力経路が異なるが、解析と可視化の中核処理は共通（ログ解析は一部異なる）である。共通に用いる要素を以下に示す。

- ログの形式：状態遷移ログと変数更新ログ（4.6 節）

- ログ解析：状態列と変数値スナップショット列の構築（4.7 節）
- 変数追跡：ウォッチ機能による変数値表示（4.8.3 節）
- 状態遷移可視化：状態遷移図の描画とハイライト表示（4.8.4 節、?? 節）
- 共通表示ロジック：ステップ番号に基づく表示更新（4.8 節）

一方、シリアル受信モードでは追加要素として、複数ポート受信が可能であり、同期タイムライン生成（同期解析の場合）を行う（?? 節）。

4.4 ファイル解析モードの設計・実装

4.4.1 ファイル解析モードの概要

ファイル解析モードは、単一ノードの動作を対象として、取得済みログの解析と可視化を行うモードである。ユーザは実行ログファイルと状態遷移図 JSON ファイルをアップロードし、ログ解析によって得られた状態列 *states* と変数値スナップショット列 *vars* を用いて、状態遷移図のハイライト表示および変数ウォッチ表示を行う。

4.4.2 ファイル解析モードの UI 設計

ファイル解析モードの UI は図 4.2 に示すように、主に以下の要素から構成される。

- (1) 実行ログファイルアップローダ
- (2) 状態遷移図 JSON ファイルアップローダ
- (3) ステップ実行制御ボタン（次へ／前へ／初期へ）
- (4) スライダーによるステップジャンプ
- (5) 変数ウォッチリスト表示エリア
- (6) 状態遷移ログ表示エリア
- (7) 状態遷移図表示エリア

ユーザは、(1)(2)により入力ファイルを選択、アップロードする。実行ログは解析され、状態遷移と変数スナップショットとして整理され、ステップ再現に用いられる。状態遷移図 JSON は状態遷移図の描画に用いられる。ユーザは (3)(4)によりステップ番号 i を操作し、(5)の変数値、(6)の状態遷移ログ、(7)の状態遷移図ハイライトを同一ステップに基づいて確認できる。

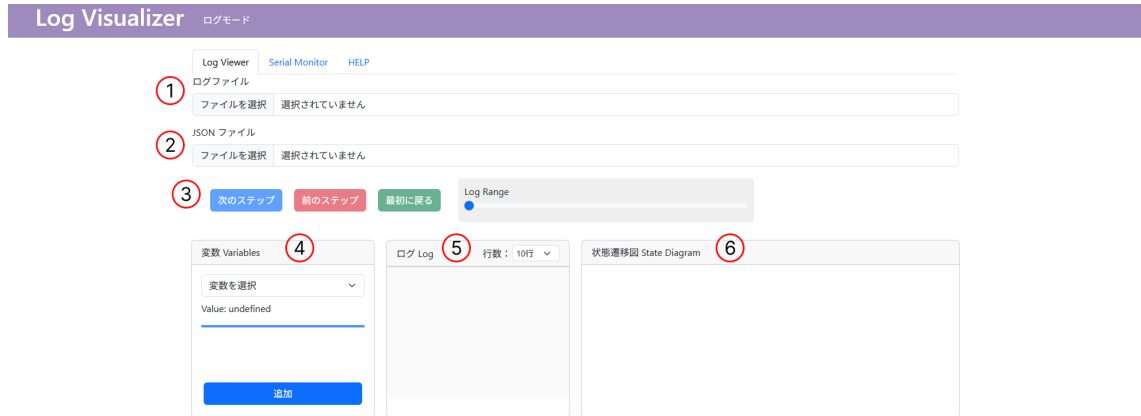


図 4.2: ファイル解析モードのレイアウト

4.5 シリアル受信モードの設計・実装

4.5.1 シリアル受信モードの概要

シリアル受信モードは、組込みデバイスを PC に接続し、シリアルポートからログを受信し、シリアルモニタ上に表示するモードである。受信ログはデバイスごとに蓄積し、ノード数と目的に応じて、**単一ノード解析**、**同期解析 (2 台)**、**マルチノード収集 (3 台以上)** をシステム側で自動的に切り替える。

単一ノード解析では、対象の 1 台のノードの受信ログを解析して *states* と *vars* を構築し、ファイル解析モードと同様に可視化デバッグを行う。同期解析では、2 台の受信ログに受信時刻を付与し、時系列を統合した同期タイムラインを生成した上で、同一時点における両ノードの状態と変数値を比較できる。マルチノード収集では、3 台以上のデバイスからログを受信し、ログの保存やコピーを主目的として扱う。(1/22 未実装)

4.5.2 シリアル受信モードのUI設計

シリアル受信モードは、ログ受信のためのシリアルモニタと、受信ログに基づいて可視化デバッグを行うUIの2つで構成される。二つのUIは同一画面内に配置され、シリアルモニタで受信したログをそのまま可視化デバッグへ引き継げるように設計されている。

シリアルモニタのUI

シリアルモニタのUIは図4.3に示すように、主に以下の要素から構成される。

- ポート選択ドロップダウン
- 接続ボタンおよび切断ボタン
- シリアルモニタ表示領域
- ログ解析ボタン（単一ノード解析用）
- 同期解析ボタン（同期タイムライン生成）



図 4.3: シリアルモニタのレイアウト

ユーザはPCと組み込みデバイスをUSB接続し、ポート選択ドロップダウンから接続するシリアルポートを選択する（図4.4）。接続ボタン押下によりログの読み取りを開始し、切断ボタン押下により読み取りを中断する。接続中は各デバイスから送信されるログをリアルタイムに受信し、シリアルモニタ表示領域に逐次表示する。

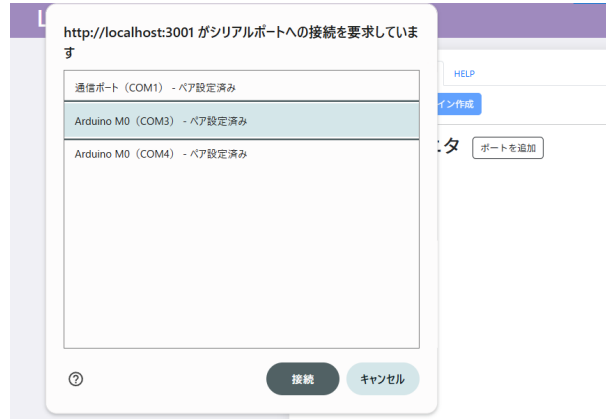


図 4.4: シリアルポート選択ドロップダウン

単一ノード解析では「ログ解析」ボタンを押下することで対象ノードの解析を実行する。同期解析では「同期解析」ボタンを押下することで2台分のログを統合し、同期タイムラインを生成する。生成された結果は可視化デバッグUIに反映され、ステップ番号 i に基づく逐次確認が可能となる。マルチノード収集では、ログ解析や同期解析は行わず、受信ログの保存・コピーを主目的とする（1/22 未実装）。

可視化デバッグのUI

シリアル受信モードの可視化デバッグUIは図4.5に示す。同期解析では、ノード1とノード2の状態遷移図表示と変数ウォッチ表示を左右に配置し、中央に同期タイムライン（統合された状態遷移ログ）を表示する。単一ノード解析では、ファイル解析モードと同じレイアウトになるようにした。

本UIでは、ステップ操作ボタンとスライダーを共通入力として、ステップ番号 i に応じて両ノードの表示を同時に更新する。これにより、ある時点における両ノードの状態と変数値を同一基準で比較できる。また、状態遷移図の描画は各ノードごとに独立して行われるため、それぞれのノードに応じた状態遷移図JSONをアップロードする必要がある。

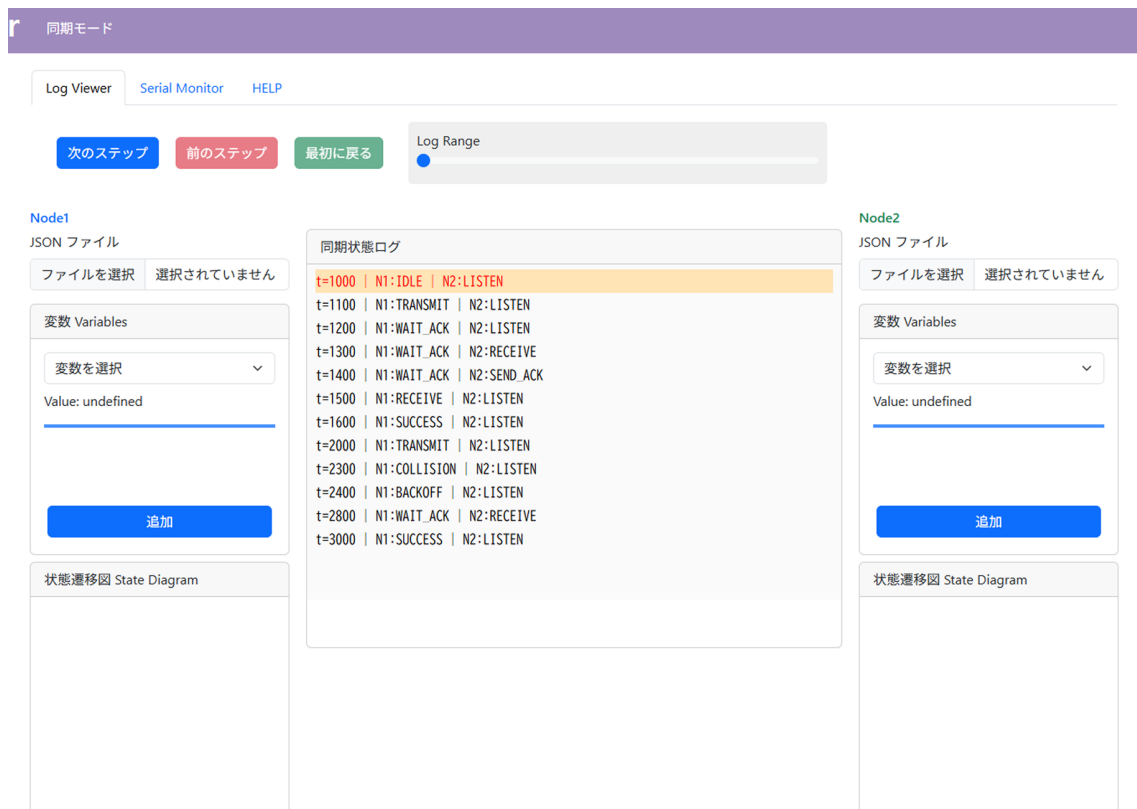


図 4.5: 同期モードのレイアウト

4.5.3 ブラウザ上でのシリアルモニタ実装

シリアル受信モードでは、専用のデスクトップアプリケーションを用いず、ブラウザ上でシリアル通信を扱えるように設計している。具体的には Web Serial API を用いることで、Web ブラウザから直接シリアルポートへ接続し、組み込みデバイスのログを受信する。ユーザは UI からポートを選択し、接続／切断を制御する。接続中は各デバイスから送信されるログをリアルタイムに受信し、逐次表示できる。なお、本方式はブラウザ環境に依存するため、利用環境の制約（対応ブラウザ・権限操作等）が存在する。

4.5.4 同期タイムラインの生成

同期モードでは、複数ノードから受信したログを同一の時間軸に統合し、ある時刻における各ノードの状態と変数値を同時に参照できるようにする。本研究では、この統合結果を**同期タイムライン**と呼ぶ。同期タイムラインは、表示上のステップ番号 i の基盤と

なるデータであり、ユーザは i を進めることで、時系列に沿って両ノードの状態と変数値を比較できる。

入力と前提

同期タイムライン生成の入力は、各ノードから受信したログ行列である。ログ行は4.6節で述べるが、状態遷移ログ (STATE:) と変数更新ログ ([VAR:]) を解析対象とする。シリアル受信モードの同期解析ではこれらに加えて、各ログ行を受信した時刻を保持する必要がある。

受信時刻に基づくタイムスタンプ付与

各ノードのログ行はブラウザ上で受信されるため、受信時点の時刻を取得してログ行に付与する。本実装では、JavaScript の `performance.now()` を用いて、コードの実行時間（ページのロード開始時刻）から実行ログ受信開始時刻までの経過時間をミリ秒単位で取得し、ログ行ごとにタイムスタンプ t を付与する。これにより、ノード間で同一の基準時刻に対してイベントを整列できる。

イベント列の統合とタイムライン時刻列の生成

次に、2台のノードの状態遷移イベント列を時刻情報に基づいてマージし、全ノード共通のタイムライン時刻列 $T = \{t_0, t_1, \dots\}$ を生成する。ここで t_i は、「いずれかのノードで状態遷移イベントが観測された時刻」である。時刻列 T を生成することで、「この時刻で他のノードはどういう状態か」を共通の添字 i で扱えるようになる。

図??の例では、ノード1が t_0 で状態 A に入り、ノード2が t_1 で状態 C に入る。以降も同様に、どちらかのノードで状態変化が発生した時刻が、タイムラインの節点として使われる。

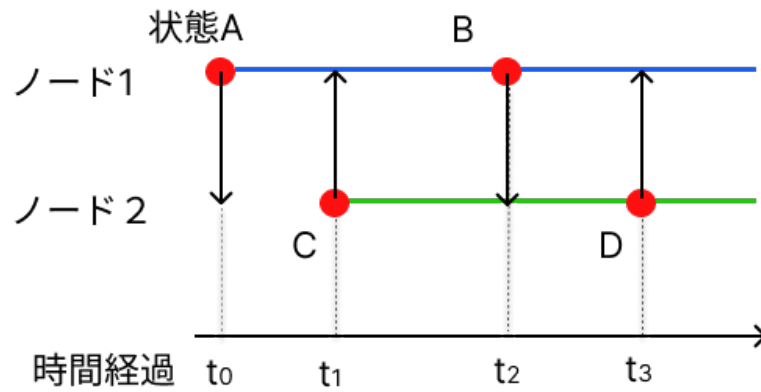


図 4.6: 同期解析におけるタイムライン (構築前)

図 4.7 では、タイムライン時刻列 $T = \{t_0, t_1, t_2, t_3\}$ に基づき、各ノードが t_i においてどの状態にあったかを記録し、同期タイムラインを構築している。例えば、ノード 1 は t_1 において図??では状態遷移ログが存在しないが、同期タイムライン構築後の図 4.7 では、直前の状態 A を引き継いで記録されている。また、ノード 2 では、 t_0 に状態遷移ログが存在しないため、状態は undefined として扱われている。

ここで重要なのは、イベントが発生していないノードも直前の値を引き継ぎ、値を保持するという点である。ある時刻 t_i でノード 1 のみにイベントがあった場合でも、ノード 2 は直前までの最新状態を引き継いで表示される。この引き継ぎにより、タイムライン上の各時刻で全ノードの状態が必ず揃い、比較が可能となる。なお、図 4.6 および図 4.7 は、状態遷移ログのみを示しているが、変数値についても同様に引き継ぎ処理を行う。

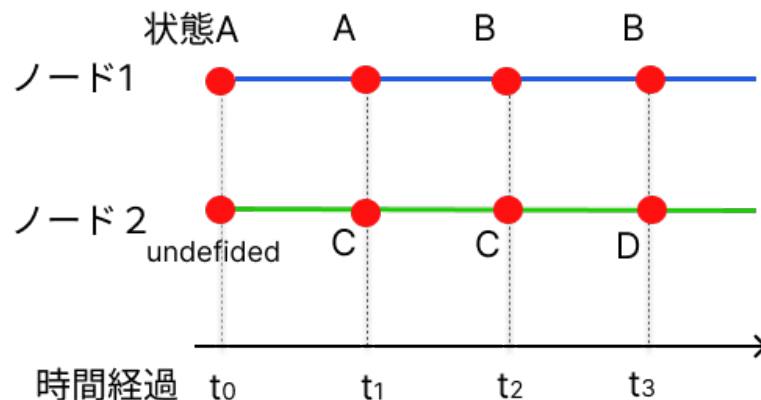


図 4.7: 構築された同期タイムラインの例

同期タイムラインのデータ構造

以上の処理により、同期タイムラインは時刻 t_i ごとのスナップショット列として表現できる。ここではサンプルログを用いて作られた同期タイムラインのデータ構造例を示す。ソースコード 4.1 にパース済みのサンプルログを示し、ソースコード 4.2 に対応する同期タイムラインのデータ構造例を示す。サンプルログでは、生データからログ解析を行い、どのノードのログか、タイムスタンプ、ログの種類、ログの内容をそれぞれ抽出し、ソースコード 4.1 の形式にしている。ここで示したサンプルログは、図??で示したノード 1 とノード 2 に類似したログを用いている。同期タイムラインでは、状態遷移が行われた $t = 0, 5, 10, 15$ の 4 つの時刻において、両ノードの状態と変数値が記録されていることが分かる。

ソースコード 4.1: パース済みサンプルログ

```
1 Node1.Logs = [  
2   { node1, time: 0, kind: "STATE", payload: { state: "A" } },  
3   { node1, time: 1, kind: "VAR", payload: { key: "var01", value: 0 } },  
4   { node1, time: 2, kind: "VAR", payload: { key: "var02", value: 0 } },  
5   { node1, time: 10, kind: "STATE", payload: { state: "B" } },  
6   { node1, time: 11, kind: "VAR", payload: { key: "var01", value: 1 } }  
7 ];  
8 Node2.Logs = [  
9   { node2, time: 5, kind: "STATE", payload: { state: "C" } },  
10  { node2, time: 6, kind: "VAR", payload: { key: "var03", value: 0 } },  
11  { node2, time: 15, kind: "STATE", payload: { state: "D" } },  
12  { node2, time: 16, kind: "VAR", payload: { key: "var03", value: 2 } }  
13 ];
```

ソースコード 4.2: 同期タイムラインのデータ構造

```
1 timeline = [  
2   {  
3     t: 0,
```

```
4     states: {
5       Node1: { state: "A", vars: {var01: 0, var02: 0} },
6       Node2: { state: "undefined", vars: {} },
7     },
8   },
9   {
10    t: 5,
11    states: {
12      Node1: { state: "A", vars: { var01: 0, var02: 0 } },
13      Node2: { state: "C", vars: {var03: 0} },
14    },
15  },
16  {
17    t: 10,
18    states: {
19      Node1: { state: "B", vars: { var01: 1, var02: 0 } },
20      Node2: { state: "C", vars: {var03: 0} },
21    },
22  },
23  {
24    t: 15,
25    states: {
26      Node1: { state: "B", vars: { var01: 1, var02: 0 } },
27      Node2: { state: "D", vars: { var03: 2} }
28    }
29  }
30 ];
```

4.6 ログ設計

4.6.1 ログ生成の設計

本節では、組込みデバイスから生成されるログの設計について述べる。ログは、状態遷移イベント、変数変更イベント、およびその他ログから構成される。デバッグ支援システムは、状態遷移イベントと変数変更イベントのみを解析対象とし、その他ログは解析時に無視する。ソースコード 4.3 にログ形式の例を示す。

状態遷移イベント

状態遷移イベントは、組込みシステムの実行中に状態が遷移した際に生成されるログであり、STATE: 状態名 の形式で表現される。状態が切り替わるたびにファームウェア内の `updateState()` 関数が呼び出され、現在の状態をログ出力する設計とした。これにより、デバッグ支援システムは受信ログから状態遷移を再現することができる。

変数変更イベント

変数変更イベントは、組込みシステム内の変数が変更された際に生成されるログであり、[VAR:<name>=<value>] の形式で表現される。変数出力には C++ のマクロ機構を用い、開発者は状態遷移図における各状態の entry 内（処理を記述するエリア）に `PRINT_VAR_LOG(変数)` と記述するだけで、変数名と値を統一形式で出力できる。これにより、デバッグ用コードの記述量を抑えつつ出力形式を統一でき、デバッグ支援システム側での解析と変数追跡を容易にする。

その他ログ

その他ログは、デバッグ情報やエラーメッセージなど、状態遷移イベントおよび変数変更イベント以外の情報を含む。これらは実行状況把握やトラブルシューティングの補助として有用であるが、本システムの状態・変数解析の対象とはしないため、解析時には無視される。

```
1 STATE: LISTEN
2 [VAR: var01=0]
3 [VAR: var02=10]
4 STATE: LISTEN
5 STATE: RECEIVE
6 [VAR: var01=1]
7 [VAR: var02=11]
8 STATE: TRANSMIT
9 [VAR: var01=2]
10 [VAR: var02=100]
```

4.7 ログ解析の設計（共通処理）

4.7.1 概要（解析の目的と処理の流れ）

本システムのログ解析は、ログ列を入力として状態遷移をステップ単位で再構成し、デバッグ表示に必要な内部情報（状態名と変数値）を整形する処理である。処理は「ログを1行ずつ読み取り、行種別（状態遷移／変数更新／その他）に応じて内部データを更新する」流れで構成される。図4.8にログ解析全体の処理フローを示す。

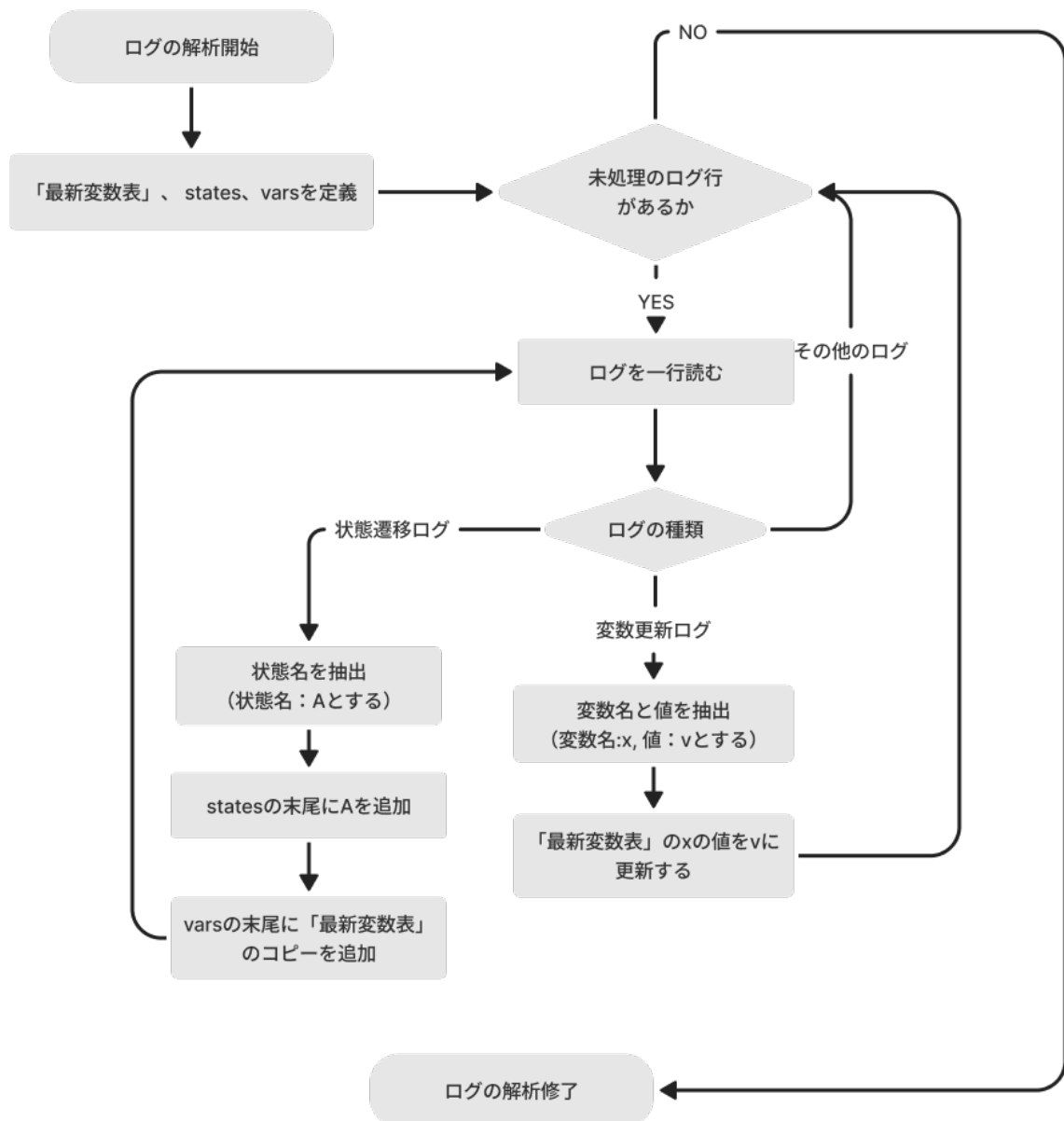


図 4.8: ログ解析の処理フロー

4.7.2 入力ログ形式とログ行の分類

ログは行単位のテキストとして扱い、各行は先頭文字列、および正規表現に基づいて以下の3種類に分類される。

- 状態遷移ログ： STATE: で始まる行
- 変数更新ログ： [VAR: で始まる行
- その他ログ：上記のいずれにも該当しない行

状態遷移ログと変数更新ログのみを解析対象とし、デバッグメッセージやエラーメッセージ等のその他ログは無視する。

4.7.3 解析結果として構築するデータ構造

本システムはログを「状態遷移単位（ステップ）」で整理するため、解析結果として状態列 *states* と 変数値スナップショット列 *vars* を構築する。状態遷移ログの出現回数をステップ番号 *i* とみなし、*states[i]* にステップ *i* 時点の状態名を、*vars[i]* にその状態における変数値集合（連想配列）を格納する。

また、直近までに観測された変数値集合を保持する連想配列を「最新変数表」と呼ぶ（実装上は *lastVars* に相当する）。最新変数表は、後述する変数値スナップショットの再構成（継承）に用いる。

4.7.4 変数値スナップショットの再構成方針（継承と更新）

変数更新ログは「値が変更された変数のみ」を出力する設計であるため、各状態において全変数の値が毎回ログに現れるとは限らない。例えば、ある状態 A で変数 *var01* と *var02* が更新されても、状態 A からの遷移先状態 B では、それらの変数が更新されない場合がある。この場合、状態 C における *var01* と *var02* の値は直前の状態である状態 A から引き継がれる必要がある。そこで本システムでは、各状態における変数値を次の規則で再構成する。

- 変数更新ログが出現した場合：最新変数表中の該当変数のみを更新する。
- 状態遷移ログが出現した場合：その時点の最新変数表をコピーし、当該ステップの変数値スナップショット *vars[i]* として確定する。

。図 4.9 に、変数値の継承（コピー）と更新の関係を示す。状態 A では *var01* が 0 に更新される（新たに登場する変数の場合でも更新と表現する）。状態 B では *var02* が 1 に更新されるが、*var01* は状態 A から引き継がれる。状態 C では *var01* が 1 に更新され、*var02* は状態 B から引き継がれる。状態 D では *var03* が 3 に更新、*var02* は 1 に更新、*var01* は直前の状態 C から引き継がれる。

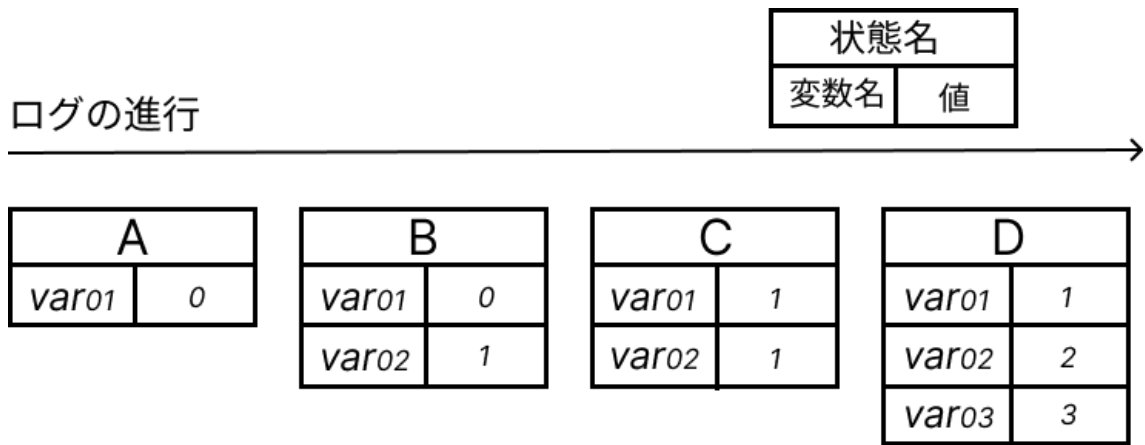


図 4.9: 変数値スナップショットの継承と更新

4.7.5 解析結果例

ログ解析によって得られる *states* と *vars* の例をソースコード 4.4 に示す。この例は、図 4.9 に示した「継承と更新」の関係に対応している。*states[i]* 時点での変数値は *vars[i]* に格納されており、各ステップでの変数値の変化が反映されていることが分かる。

ソースコード 4.4: ログ解析結果の例 (*states* と *vars*)

```

1 states = ["A", "B", "C", "D"]
2
3 vars = [
4     { var01: 0 },                // A のスナップショット
5     { var01: 0, var02: 1 },      // B のスナップショット
6     { var01: 1, var02: 1 },      // C のスナップショット
7     { var01: 1, var02: 2, var03: 3 } // D のスナップショット
8 ]

```

4.8 ステップ番号に基づく共通表示ロジック

4.8.1 共通の前提：ステップ番号 i と状態スナップショット

本システムでは、ログ解析により構築された状態列 *states[i]* と変数値スナップショット

ト列 $vars[i]$ (4.7節) を基盤として、各種表示を統一的に制御する。ここで i は「状態遷移ログの出現順」に対応するステップ番号であり、ユーザがスライダー操作やボタン操作を操作すると動的に i が変更され、UI は $states[i]$ および $vars[i]$ を参照して表示を更新する。この設計により、ファイル解析モードとシリアル受信モードで入力経路が異なる場合でも、「ステップ番号 i を中心に表示を駆動する」という共通構造を維持できる。

4.8.2 ステップ実行のロジック

ステップ実行機能は、ステップ番号 i を1ずつ増減させることで、状態遷移の進行を逐次確認できるようにする機能である。ユーザ操作として、(1) 次のステップへ、(2) 前のステップへ、(3) 最初にもどる、およびスライダーによる任意ステップへの移動を提供している。

内部的には現在のステップ番号を i とし、 $0 \leq i \leq (N - 1)$ (N は状態列の長さ) を満たすよう境界条件を設ける。ユーザ操作により i が更新されると、状態遷移ログ表示、変数ウォッチ表示、状態遷移図のハイライト表示を同時に更新する。

4.8.3 変数ウォッチ機能のロジック

変数ウォッチ機能は、ユーザが監視したい変数（ウォッチリスト）を任意の数指定し、現在ステップ i における変数値 $vars[i]$ を参照して表示する機能である。ログは「変更された変数のみ」が出力されるが、ログ解析段階で変数値スナップショットを再構成しているため (4.7節)、表示側では単純に $vars[i]$ を参照するだけでよい。

未到達の変数を監視対象にした場合、その変数の値は登場するまで *undefined* と表示する。

4.8.4 Mermaid による状態遷移図描画

状態遷移図の描画には Mermaid.js (以下 Mermaid) を用いる。Mermaid は状態遷移図を特定のテキスト（構文）として記述し、その文字列から図を生成するため、本システムでは状態遷移図 JSON (状態集合・遷移集合・初期状態の情報を持つ) から Mermaid

構文文字列を生成する。さらに、ステップ i に応じて現在状態をハイライトするスタイル指定を付加することで、ログと状態遷移図を連動させた表示を実現する。

工夫点として、状態の自己ループの場合は、矢印を自状態に向けるのではなく、通常のハイライトとは違うスタイルを適用している。これは、自己ループの矢印を状態遷移図内に描画すると Mermaid が出力する状態遷移図のレイアウトが崩れてしまうためである。

構文生成は概ね次の手順で行う。(1) ユーザによって状態遷移図 JSON がアップロードされる (2) JSON をパースして状態集合・遷移集合・初期状態を抽出する (3) Mermaid の状態遷移図の構文に基づいて文字列を生成する

状態遷移図の JSON 例をソースコード??に、Mermaid 構文生成の擬似コードをソースコード 4.6 に示す。

JSON では、状態集合 `states`、遷移集合 `transitions`、および初期状態 `initialState` を定義している。この情報を基に、Mermaid 構文を生成する。

生成された Mermaid 構文では、状態遷移図の方向を上から下に指定する `direction TB` を用いている。また、初期状態 (*) から A への遷移を定義し、その後その他の遷移についても定義している。加えて、最終行でステップ i に応じて現在状態をハイライトするスタイル指定を付加している。これにより、状態遷移図上で現在状態が強調表示される。

ソースコード 4.5: 状態遷移図の JSON 例

```
1 {
2   "states": ["A", "B", "C"],
3   "transitions": [
4     { "from": "A", "to": "A" },
5     { "from": "A", "to": "B" },
6     { "from": "B", "to": "C" },
7     { "from": "C", "to": "A" }
8   ],
9   "initialState": "A"
10 }
```

ソースコード 4.6: 生成された Mermaid 構文

```
1 stateDiagram-v2
2 direction TB
3 [*] --> A
4 A --> A
5 A --> B
6 B --> C
7 C --> A
8 style state[i] fill:#f9f,stroke:#333,stroke-width:2px;
```

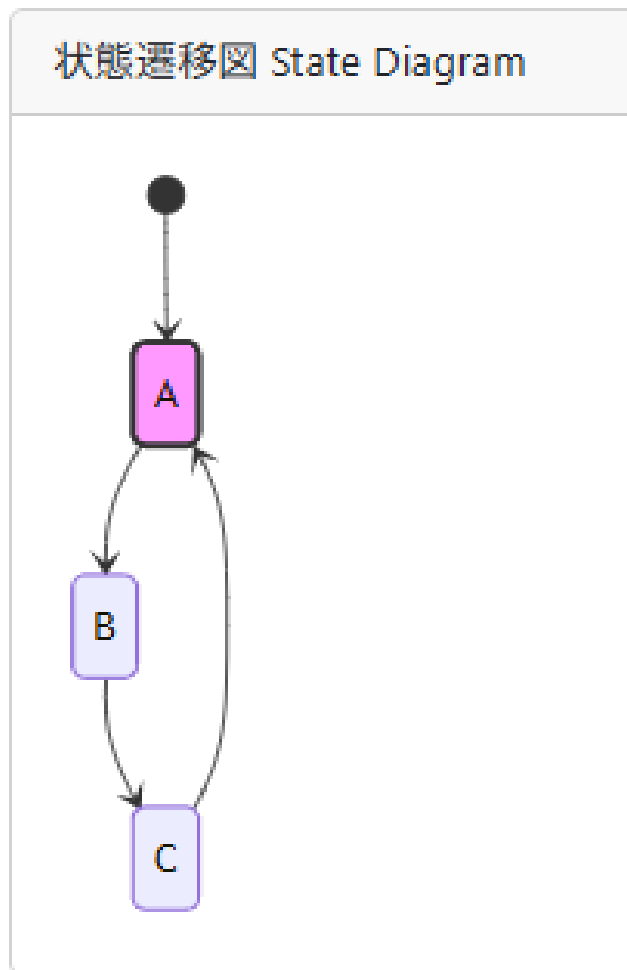


図 4.10: Mermaid によって描画された状態遷移図の例

以上のように、本システムではステップ番号 i を中心に、状態列 $states[i]$ と 変数ス

ナップショット *vars[i]* を参照して、各表示（ログ表示・ウォッチ表示・状態遷移図表示）を一貫して更新する構造として設計した。

第 5 章

評価

5.0.1 評価の目的

本章では、本研究で開発したデバッグ支援システムについて、提案機能が実機ログ解析において有効に機能し、その有用性を確認する。特に、(1) 状態遷移の追跡、(2) 変数値追跡、(3) ステップ実行に基づく逐次確認、および (4) (シリアル受信モードの同期解析における) 2 ノードの比較可能性の観点から、実ログを用いて動作例を示し、定性的に評価する。ただし、本研究で対象とする実機上で動作させるプロトコルは、無線環境の競合やタイミング揺らぎ等の外的要因により、特定の状態遷移を狙って発生させることが難しく、同一のログを高い再現性で取得することが困難である。そこで本章では、実機実装が出力するログ形式 (STATE/VAR などのイベント種別, 出力規則, 順序制約) に準拠しつつ、提案機能の挙動が読者に明確に伝わるように、実行ログを整形した例示ログを用いて動作例を示す。

5.0.2 評価対象

評価対象は、本研究で開発したデバッグ支援システムの二つのモード (ファイル解析モード、シリアル受信モード) の機能である。ファイル解析モードでは、単一のノードから得られた実行ログファイルを入力とし、状態遷移の追跡、変数値の追跡、ステップ実行機能の評価する。一方、シリアル受信モードでは、2 台のノードから同時に受信した実行ログを用いて同期解析を行い、2 台のノードの状態・変数値を同一時刻で比較できることを評価する。

- 単一ノードデバッグ（共通部分）
 - － 状態遷移の可視化（状態遷移図のハイライトとログ表示）
 - － 変数ウォッチ機能（状態ステップに対応した変数値追跡）
 - － ステップ実行および任意ステップへの移動（スライダー）
- 同期モード固有機能（差分）
 - － 2台のノードログを統合した同期タイムライン表示
 - － 同一時刻における状態・変数の比較

5.0.3 評価で用いるプロトコル

評価には、実機で動作する無線通信プロトコルの実行ログと、対応する状態遷移図（JSON）を用いる。実機で動作させるプロトコルは、pure ALOHA ベースの簡易無線通信プロトコルであり、ノードは送信、受信、待機、衝突検出、バックオフなどの状態を遷移しながら通信を行う。各ノードは、状態遷移イベントと任意の変数更新イベントをログとして出力し、本システムはこれらのログを解析して状態遷移の追跡と変数値の監視を行う。

5.1 ファイル解析モードにおけるデバッグの評価

5.1.1 評価シナリオ

本節では、単一ノードの実行ログ（.log ファイル）を入力として、状態遷移の進行および変数値の推移をステップ単位で追跡できることを確認する。

5.1.2 ログ入力と解析結果の生成（states/vars）

評価に用いる入力ログの抜粋を 5.1 に示す。本ログは、状態遷移ログ（STATE:）と変数更新ログ（[VAR:）から構成され、待機状態の継続（同一状態の連続出現）やパケットの送受信、衝突、バックオフを含む実機上で起こりうる典型的な遷移を表している。ロ

グ入力後、システムはログ解析を行い、状態列 *states* と、各ステップに対応する変数スナップショット列 *vars* を構築し、以降の表示更新はこれらの配列インデックス（ステップ番号）に基づいて行われる。

ソースコード 5.1: デバッグシステムに入力するログ（抜粋）

```
1 STATE: IDLE
2 [VAR:DEVICEEI=1]
3 STATE: TRANSMIT
4 [VAR:success_cnt=0]
5 STATE: WAITACK
6 STATE: WAITACK
7 STATE: RECEIVE
8 [VAR:ack=1]
9 [VAR:success_cnt=1]
10 STATE: SUCCESS
11 STATE: IDLE
12 [VAR:DEVICEEI=1]
13 STATE: TRANSMIT
14 [VAR:success_cnt=1]
15 STATE: WAITACK
16 STATE: WAITACK
17 STATE: WAITACK
18 STATE: COLLISION
19 STATE: BACKOFF
20 STATE: BACKOFF
21 STATE: TRANSMIT
22 [VAR:success_cnt=1]
23 STATE: WAITACK
24 STATE: RECEIVE
25 [VAR:ack=1]
```

```
26 [VAR: success_cnt=2]
27 STATE: SUCCESS
```

ログ解析により得られる状態列の抜粋をソースコード 5.2 に示す。同一状態が連続する場合も、ログ出現順に状態名が列として保持されるため、UI 側では当該インデックスを「現在ステップ」として扱い、待機の継続等をステップ単位で追跡できる。これにより、状態遷移図上で、同一状態が継続していることを特定のハイライトスタイルで表示できる。

ソースコード 5.2: ログ解析結果の例 (states の抜粋)

```
1 states = [
2     "IDLE",
3     "TRANSMIT",
4     "WAITACK",
5     "WAITACK",
6     "RECEIVE",
7     "SUCCESS",
8     "IDLE",
9     "TRANSMIT",
10    "WAITACK",
11    "WAITACK",
12    "WAITACK",
13    "COLLISION",
14    "BACKOFF",
15    "BACKOFF",
16    "TRANSMIT",
17    "WAITACK",
18    "RECEIVE",
19    "SUCCESS",
```

20]

変数スナップショット列 *vars* の抜粋をソースコード 5.3 に示す。本研究のログ設計では、[VAR:...] は「変更された変数のみ」を出力するため、ログに現れない変数については直前値を継承し、各ステップで参照可能な形に再構成する（4.7.4 節参照）。この再構成により、状態遷移 (*states*) と変数値 (*vars*) を同一ステップに対応付けて表示できる。ソースコード 5.2 の *i* 行目の状態（ステップ）に対応する変数スナップショットはソースコード 5.3 の *i* 行目に対応している。

ソースコード 5.3: ログ解析結果の例 (*vars* の抜粋)

```

1 vars = [
2   {"DEVICE_ID": "1"},
3   {"DEVICE_ID": "1", "success_cnt": "0"},
4   {"DEVICE_ID": "1", "success_cnt": "0"},
5   {"DEVICE_ID": "1", "success_cnt": "0"},
6   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
7   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
8   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
9   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
10  {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
11  {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
12  {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
13  {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
14  {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
15  {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
16  {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
17  {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
18  {"DEVICE_ID": "1", "success_cnt": "2", "ack": "1"},
19  {"DEVICE_ID": "1", "success_cnt": "2", "ack": "1"}
20 ]

```

5.1.3 状態遷移ログ表示

ここでは、状態遷移ログ表示領域において、ログ解析により構築された状態列 *states* を基に状態遷移を表示し、ユーザが現在の状態遷移位置を把握できることを確認する。

図 5.1 に表示例を示す。状態遷移ログ表示領域では、ログ解析により構築された状態列 *states* (5.2) を基に、状態遷移を表示する。ステップ番号 *i* に対応する状態 *states*[*i*] はハイライト表示され、ユーザが現在の状態遷移位置を把握できるようになっている。また、状態遷移ログ内の任意のログ行をクリックすると、そのステップ番号 *i* にジャンプし、他の表示（変数ウォッチ表示、状態遷移図表示）も同時に更新される。加えて、ユーザは表示行数として 10 行、30 行、50 行、100 行を選択できるため、長いログに対しても適切な表示範囲を選択できる。連続して同一状態が出現する場合（自己遷移や待機状態の継続）においても、ステップ番号（インデックス）の進行に従って、状態列 *states* の該当要素がログ上の現在位置として反映される。これにより、「同じ状態が続いている」ことをステップ単位で追跡できることを確認した。

以上より、本機能により、状態遷移を逐次的に可視化し、ユーザが状態遷移の進行を把握できることを示した。

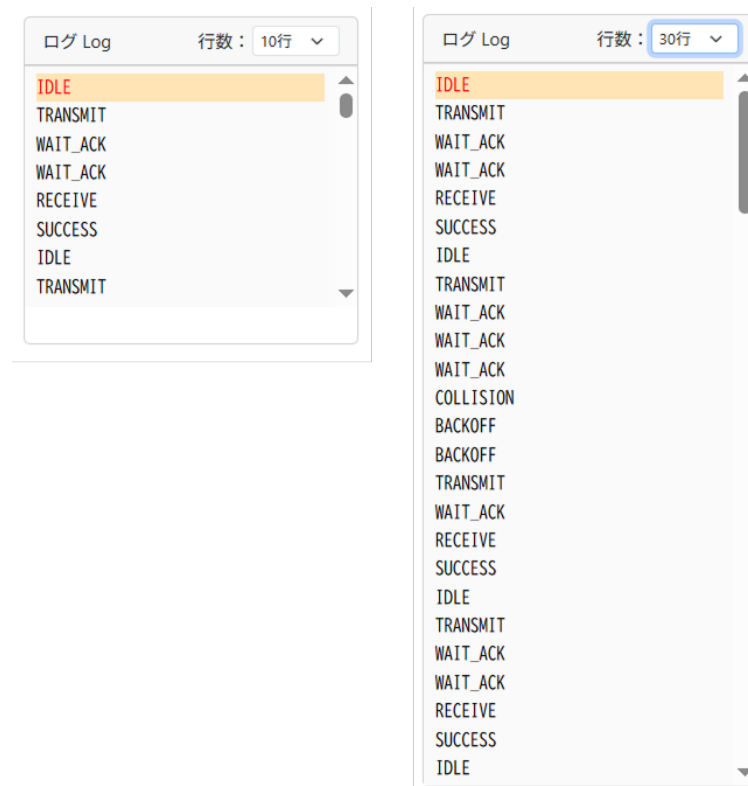


図 5.1: 状態遷移ログ表示（左：10 行表示、右：30 行表示）

5.1.4 変数ウォッチ表示（状態ステップに対応した変数値追跡）

変数ウォッチエリアでは、ユーザが監視対象とする変数を選択し（図 5.2）、選択された変数の値をステップ番号に対応付けて表示する（図 5.3）。左の画像では *DEVICE_EI*、*success_cnt* と *ack* を選択しており、右の画像では状態の進行後ステップ番号に基づいて変数値が変化している様子を示している。この表示は、ログ解析で生成される変数スナップショット列 *vars*（5.3）を参照して行われ、各ステップにおける変数値を一貫した形式で確認できる。

本研究のログ設計では、[VAR:...] が「変更された変数のみ」を出力するため、ログ上で値が現れないステップにおいても直前値を継承することで、各ステップのスナップショットを再構成している。その結果、ユーザはログ行を手作業で追跡・照合することなく、状態列 *states* と同一ステップ上で変数値の推移を追跡できることを確認した。

以上より、本機能により「状態遷移の進行」と「複数の変数値の変化」を並行して同一ステップで対応付けられ、デバッグ時の確認作業を簡略化できることを示した。



The image shows a dialog box titled "変数 Variables". It contains a dropdown menu labeled "変数を選択" (Select variable) with a downward arrow. Below the dropdown is a list of variables: "変数を選択", "DEVICE_EI", "success_cnt", and "ack". The "DEVICE_EI" variable is currently selected and highlighted in blue. At the bottom of the dialog is a blue button labeled "追加" (Add).

図 5.2: 変数ウォッチ表示（変数選択）

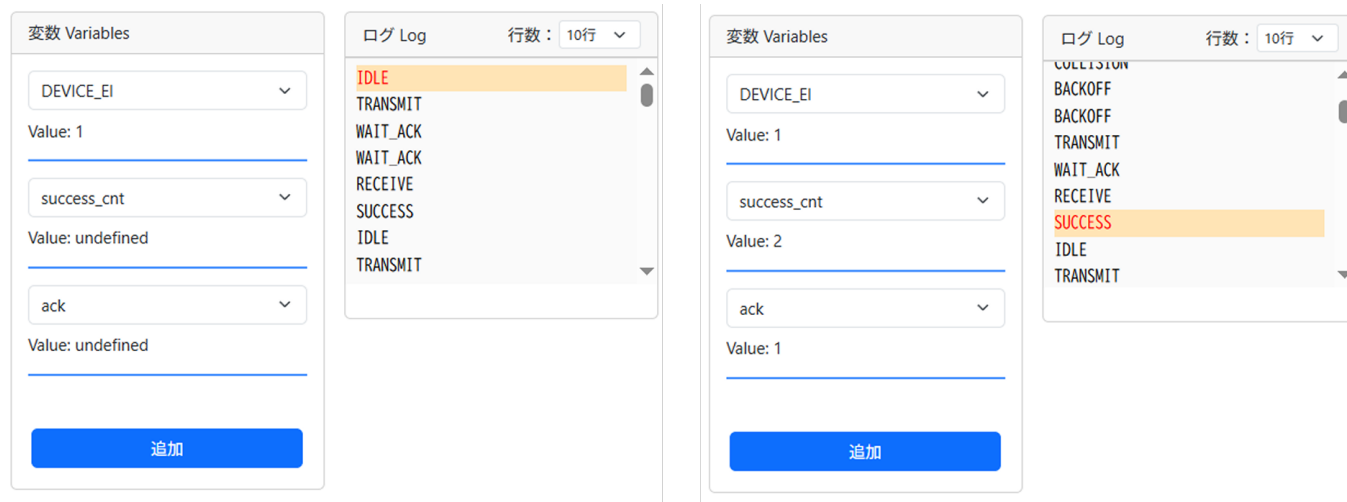


図 5.3: 変数ウォッチ表示（ステップによる変数の変化）

5.1.5 状態遷移図表示（JSON 入力→Mermaid 変換→描画）

本研究で用いるプロトコルは、状態遷移図ベースであり、その可視化はデバッグにおいて重要である。本システムにおける状態遷移図の描画は、状態遷移図 JSON を入力として Mermaid 構文（Mermaid が状態遷移図を描画するための記法）へ変換し、その結果を描画することによって実現している。

Pure ALOHA の状態遷移図 JSON 例をソースコード 5.4 に示す。states 配列に状態名を、transitions 配列に遷移関係を、initialState に初期状態を定義している。この JSON から生成された Mermaid 構文をソースコード 5.5 に示す。最下行で現在状態のスタイル指定（ハイライト）を付加している。

ソースコード 5.4: 状態遷移図 JSON の例（Pure ALOHA）

```

1 {
2   "states": [
3     "IDLE",
4     "TRANSMIT",
5     "WAITACK",
6     "SUCCESS",
7     "COLLISION",
8     "BACKOFF"

```

```
9    ],
10    "transitions": [
11      {"from": "IDLE", "to": "TRANSMIT"},
12      {"from": "TRANSMIT", "to": "WAITACK"},
13      {"from": "WAITACK", "to": "SUCCESS"},
14      {"from": "WAITACK", "to": "COLLISION"},
15      {"from": "WAITACK", "to": "WAITACK"},
16      {"from": "SUCCESS", "to": "IDLE"},
17      {"from": "COLLISION", "to": "BACKOFF"},
18      {"from": "BACKOFF", "to": "TRANSMIT"},
19      {"from": "BACKOFF", "to": "BACKOFF"}
20    ],
21    "initialState": "IDLE"
22  }
```

ソースコード 5.5: 生成された Mermaid 構文 (Pure ALOHA)

```
1 stateDiagram-v2
2   direction TB
3   [*] --> IDLE
4   IDLE --> TRANSMIT
5   TRANSMIT --> WAITACK
6   WAITACK --> SUCCESS
7   WAITACK --> COLLISION
8   SUCCESS --> IDLE
9   COLLISION --> BACKOFF
10  BACKOFF --> TRANSMIT
11  style state[i] fill:#f9f,stroke:#333,stroke-width:2px;
```


図 5.4 に、Mermaid による状態遷移図の描画例を示す。状態遷移図が表示されることで、ログの状態列（5.2）が図上の状態遷移として対応付けられ、前後の状態などの関係性を視覚的に把握できることを確認した。

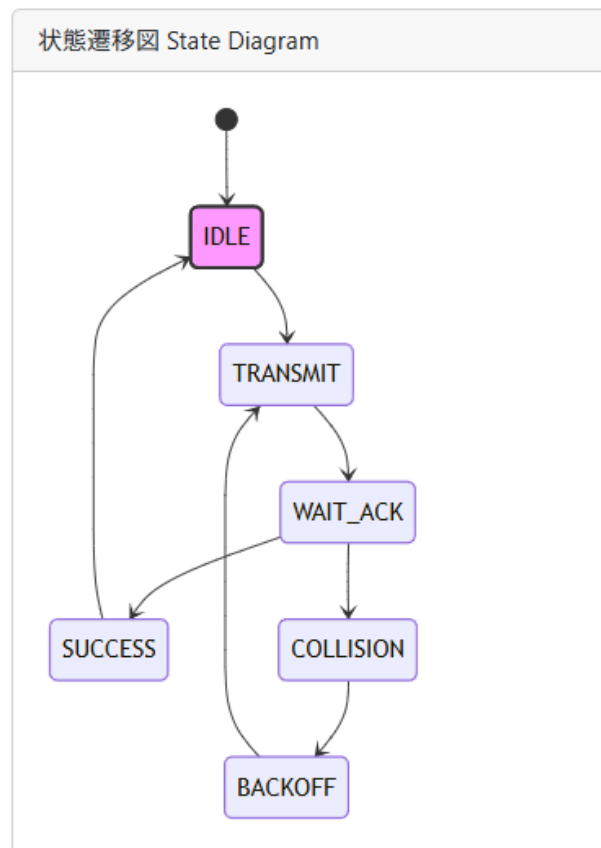


図 5.4: Mermaid による状態遷移図の描画例

5.1.6 ステップ操作・スライダ操作（表示要素の同期更新）

次ステップボタン・前ステップボタンの操作により、状態遷移図表示、状態遷移ログ表示、変数ウォッチ表示が、同一のステップ番号（インデックス）に基づいて同期して更新されることを確認する。図 5.5 に、ステップ実行により表示要素が連動して更新される例を示す。中央の状態遷移表示では、現在ステップに対応する状態（*TRANSMIT*）がハイライトされている。右側の状態遷移図では、現在の状態が同様にハイライトされている。左側の変数ウォッチ表示では、現在ステップに対応する変数値が正しく表示されている。これらの表示要素は、ステップ実行により同一インデックスを参照して更新されるため、一貫した情報提示が行われていることを確認した。

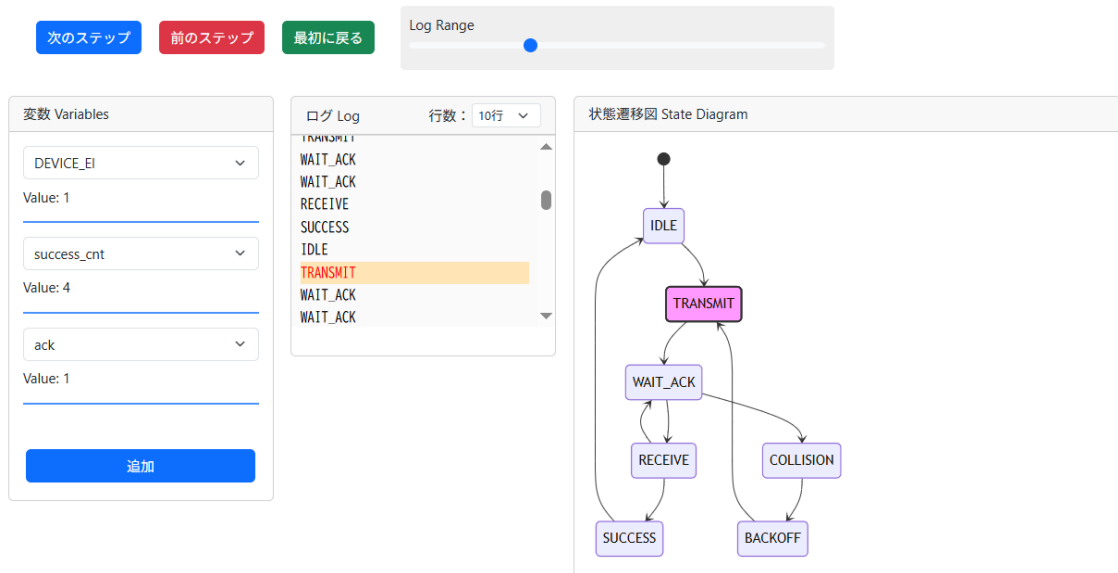


図 5.5: ステップ実行の様子

また、スライダ操作により任意ステップへ移動した場合も同様に、状態列 *states* (5.2) および変数スナップショット列 *vars* (5.3) の同一インデックスを参照することで、各表示要素が整合して更新されることを確認した。この一貫したインデックス参照により、「状態遷移図」「ログ」「変数」の対応関係を崩さずに任意位置へジャンプでき、逐次デバッグの効率化に寄与する。

5.2 シリアル受信モード固有機能の評価（2 ノード同期タイムライン）

5.2.1 評価シナリオ

本節では、シリアル受信モード特有の機能である、シリアルモニタを用いたログの受信と2ノードの同期タイムライン生成と表示について評価を行う。組み込みデバイスの構成としては、2ノード構成（中継器1台＋端末2台）で確認する。なお、端末2台のうち1台は省略し、中継器ともう1台の端末ノードのログを用いた同期解析を行う。図5.6に評価に用いるデバイス構成を示す。2台の端末を中継器に接続しているため、必ずしも対象の端末の通信が成功するとは限らず、衝突や再送が発生する状況を再現する。

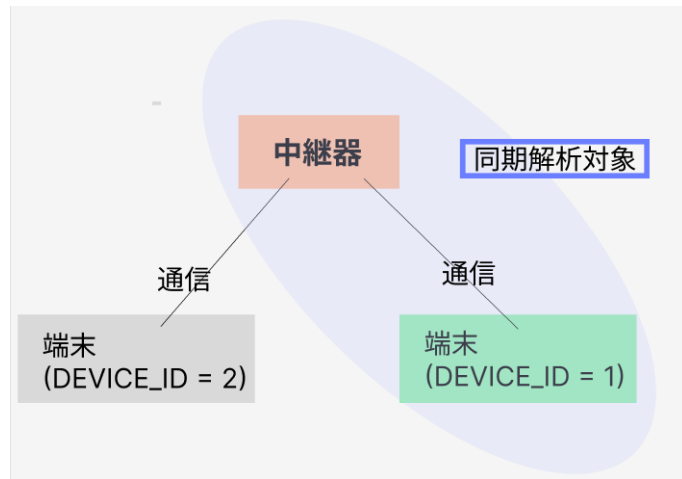


図 5.6: 評価に用いるデバイス構成

具体的に評価する項目は、(i) シリアルモニタが各デバイスから状態遷移ログおよび変数ログを継続的に受信できること、(ii) 受信ログを時刻情報に基づき統合して同期タイムラインを構築できること、(iii) タイムライン上の任意時刻において両ノードの状態・変数が対応付けて提示され、送受信関係や待機の重なり等の時間的關係が読み取れること、の3点とする。

5.2.2 シリアルモニタによるログ受信

まず、シリアルモニタが各デバイスからのログをデバイス別に分離して表示できることを確認する。図 5.7 は、中継器（左）と端末（右）からのログが同時に流入する状況で、状態遷移（STATE: ...）と変数出力（[VAR: key=value]）が時刻付きで蓄積される様子を示めている。ここで重要なのは、後続の同期処理に必要な「時刻」「イベント種別（状態/変数）」「値」が欠落、混在することなく、2 ノード並行して取得できている点である。また、従来研究では、別のアプリケーション上のシリアルモニタ（Visual Studio Code の PlatformIO 拡張機能内のシリアルモニタ）を用いて同様のログ受信を行っていた。この場合、複数の組み込みデバイスのログを並行して取得することができなかったのが、デバッグ効率を下げる一つの要因となっていた。本システムではのデバッグシステム上にシリアルモニタを Web Serial API を用いて実装し、複数ノードからのログを同時に受信・表示でき、そのログをそのままデバッグへ移行できる点がデバッグの効率化に寄与することを確認した。

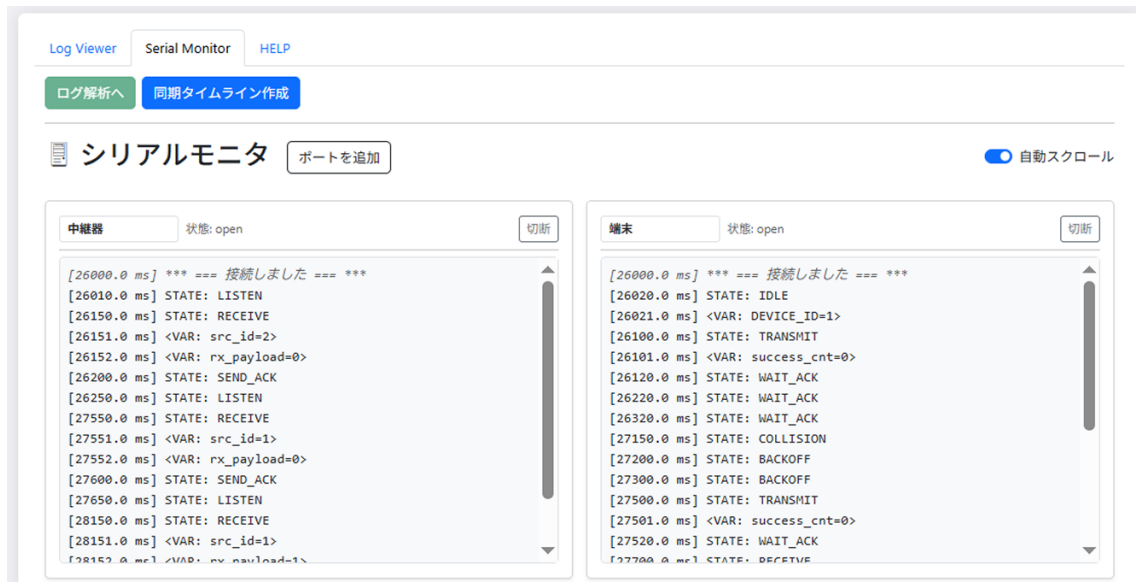


図 5.7: シリアルモニタによるログ受信の様子（左：中継器、右：端末）

5.2.3 同期タイムライン生成と表示

次に、2 台のノードの受信ログを統合して同期タイムラインとして再構成できることを確認する。同期タイムラインは、各ノードで観測されたイベント列を時刻順に統合し、共通の時間軸上で「ある時刻における各ノードの状態と変数値スナップショット」を同時に保持するデータ構造として生成される（4.5.4 節参照）。シリアルモニタでログを受信した後、ユーザが「同期解析」ボタン（1/22 検討中）を押下することで、ログ解析とタイムライン構築が実行され、結果が可視化される。

ソースコード 5.6 に、生成された同期タイムラインの例を示す。4.5.4 節で紹介したように、タイムラインは各時刻 t における各ノードの状態と変数値を含むオブジェクトの配列として表現される。

図 5.8 に、同期化された状態遷移表示例を示す。本表示により、中継器と端末のイベントが単一の時系列として整列され、時刻 t に対応する両ノードの状態が同時に提示されることが分かる。変数ウォッチ機能については、単一ノードの場合と同様のため、ここでは省略する。これにより、2 台のノードが同時点でどのような状態にあり、どのような変数値を持っているかを一目で把握できる。

ソースコード 5.6: 生成された同期タイムライン（抜粋）

```
1 timeline = [  
2     {  
3         t: 1000,  
4         states: {  
5             Node1: { state: "LISTEN", vars: {} },  
6             Node2: { state: "IDLE", vars: { DEVICEID: 1 } },  
7         },  
8     },  
9     {  
10        t: 1100,  
11        states: {  
12            Node1: { state: "RECEIVE", vars: { src_id: 2, rx_payload: 0 } },  
13            Node2: { state: "TRANSMIT", vars: { DEVICEID: 1, success_cnt: 0 } },  
14        },  
15    },  
16    {  
17        t: 1200,  
18        states: {  
19            Node1: { state: "SENDACK", vars: { src_id: 2, rx_payload: 0 } },  
20            Node2: { state: "WAITACK", vars: { DEVICEID: 1, success_cnt: 0 } },  
21        },  
22    },  
23    {  
24        t: 1300,  
25        states: {  
26            Node1: { state: "LISTEN", vars: { src_id: 2, rx_payload: 0 } },  
27            Node2: { state: "WAITACK", vars: { DEVICEID: 1, success_cnt: 0 } },  
28        },  
29    }  
30 ];
```



t=1000	N1:LISTEN	N2:IDLE
t=1100	N1:RECEIVE	N2:TRANSMIT
t=1200	N1:SEND_ACK	N2:WAIT_ACK
t=1300	N1:LISTEN	N2:WAIT_ACK
t=2100	N1:LISTEN	N2:COLLISION
t=2200	N1:LISTEN	N2:BACKOFF
t=2400	N1:LISTEN	N2:TRANSMIT
t=2500	N1:RECEIVE	N2:WAIT_ACK
t=2600	N1:SEND_ACK	N2:WAIT_ACK
t=2700	N1:LISTEN	N2:RECEIVE
t=2800	N1:LISTEN	N2:SUCCESS
t=3000	N1:LISTEN	N2:TRANSMIT
t=3100	N1:RECEIVE	N2:WAIT_ACK
t=3200	N1:SEND_ACK	N2:WAIT_ACK

図 5.8: 同期化された状態遷移表示例

5.2.4 同期解析の有効性（定性的観察）

最後に、同期タイムラインによって2ノード間の相互作用を把握しやすくなる点を定性的に確認する。図5.9は、同期タイムライン上の特定ステップに到達したときの、同期状態遷移ログおよび両ノードの状態遷移図および変数表示が同時に更新される様子を示す。 $t = 2600$ のステップで、中継器はSEND_ACK状態にあり、端末はWAIT_ACK状態にあることが分かる。次のステップでは、端末が何らかのパケットを受信し、RECEIVE状態に遷移している。その後SUCCESS状態に遷移し、端末の変数success_cntが1になっている（初期値は0）。これから、端末は中継器からのACKパケットを正しく受信し、通信が成功したことが読み取れる。このように、同期タイムラインにより、同期化された状態遷移と変数値の追跡が可能となり、ノード間の送受信関係が把握しやすくなることを確認した。

また、別の例では端末がWAIT_ACKを継続した後にCOLLISIONへ遷移する場合、中継器側に該当端末由来の受信（例：src_id=1）が存在しないつまり該当端末からのパケットが届いていない、あるいは別端末由来の受信（例：src_id=2）が優先して観測される、といった関係を時系列として追跡できる。このように、同期タイムラインは「同一時刻

の状態・変数の突き合わせ」を可能にし、単一ノードのログ閲覧では把握しにくい送受信の因果関係や待機時間の重なりを解釈しやすくする。

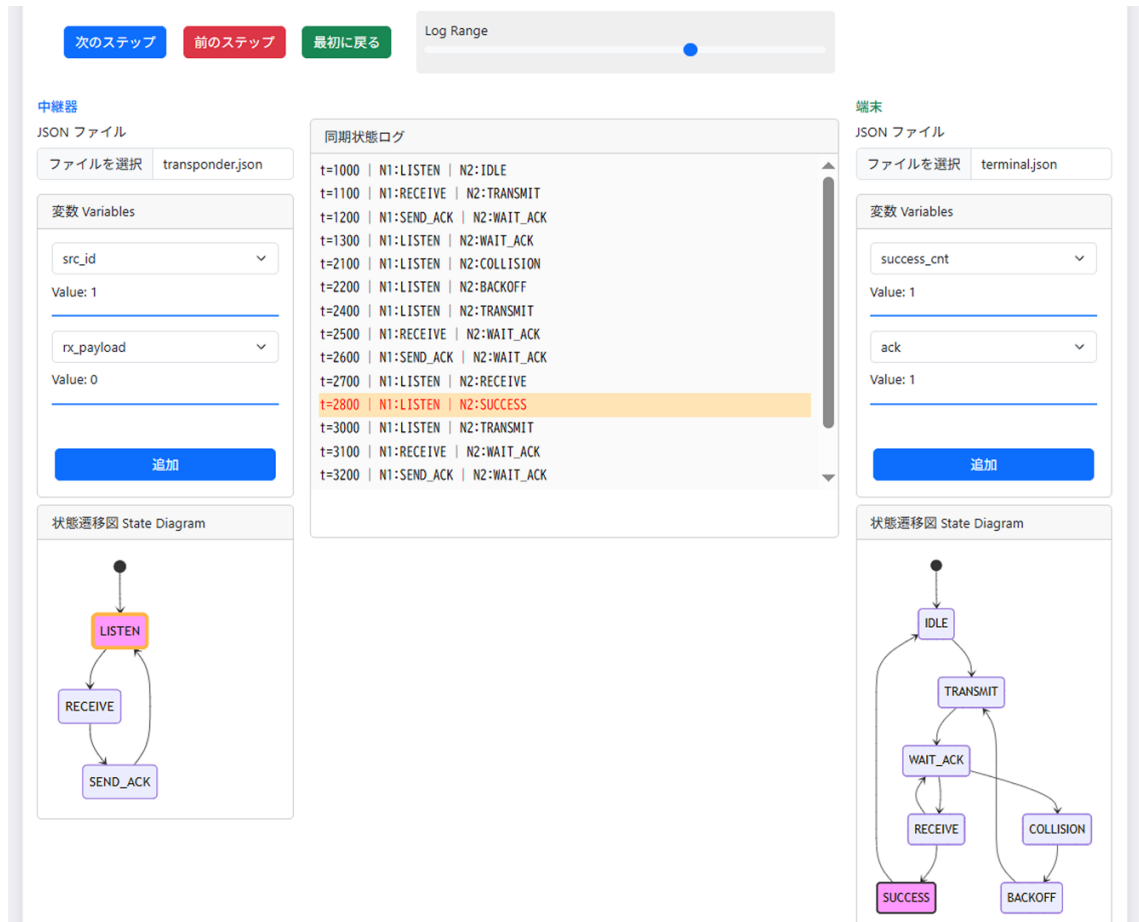


図 5.9: 同期デバッグ実行例（タイムライン選択に連動した複数ノードの同時表示）

5.3 評価のまとめ

本章では、実機実装のログ形式に準拠した例示ログを用い、提案するデバッグ支援システムの各機能がログ解析に基づいて整合的に動作することを定性的に確認した。ファイル解析モードにおいては、状態列 *states* と変数スナップショット列 *vars* を同一ステップで対応付けることにより、状態遷移の進行と複数変数の変化を並行して追跡できることを示した。さらに、状態遷移図・ログ・変数表示が同一インデックスで連動して更新されるため、従来のようなログ行の手作業照合に比べて、実行の流れを保ったまま原因箇所を探索しやすくなる。また、シリアル受信モードでは、複数ノードからのログを同時に受信し、時刻情報に基づいて共通時間軸へ統合する同期タイムラインを構築できるこ

とを示した。これにより、同一時刻における状態・変数の突き合わせが可能となり、送受信関係や待機の重なり、競合・再送といった分散実行特有の現象を、複数ノードの対応関係として解釈しやすくなることを確認した。

一方で、本章の評価は機能成立の確認を主目的とした定性的評価であり、作業時間の短縮量や原因特定の精度向上といった効果を定量的に示すには至っていない。また、同期タイムラインの有効性は、ログの時刻精度やイベント欠落の有無など、入力ログの品質にも依存する。さらに、本章の2ノード同期は、同一PCに接続された各デバイスのログ受信時刻を共通基準として用いる前提に基づくため、地理的に離れたデバイス間で同等の時刻整合を保証する用途には、そのまま適用できない。これらの点を踏まえ、次章では、本章で得られた観察結果を基に、提案方式の有効範囲・限界・適用上の注意点を整理し、今後の改善方針（例：評価指標の設計、入力ログの信頼性向上、UI・解析手順の拡張）について考察する。

第 6 章

考察

6.1 既存手法とのギャップと，本方式が埋めた点

従来のデバッグは，シリアルコンソールへ出力されたログを時間順に追い，開発者が一行ずつログを確認し、原因箇所を特定する方法で行われてきた。この方法は柔軟である一方，通信処理が高速かつ複雑に進行する場合，ログの量と状態遷移の分岐の多さにより，それぞれの処理の関係性を把握しにくいという課題がある。特に，分散システムにおいては，複数ノード間の相互作用を理解する必要があり，単一ノードのログを時間順に追うだけでは，原因特定が困難になる場合が多い。

6.1.1 「時間順の羅列」から「状態遷移という意味単位」へ

本方式では，状態遷移ログを基準としてタイムラインを構成し，ログ列を「状態遷移の進行」という意味単位で再構成する。これにより，ユーザはステップ実行ボタンおよびスライダを用いて任意のステップへ直接アクセスでき，注目箇所の再現や前後状態との比較が可能となる。つまり，単なるログ閲覧ではなく，「状態の進行を操作する」という形へユーザ体験を変換した点が，本方式の有効性を支える要因である。

6.1.2 「ログ行の追跡」から「同時更新による照合」へ

状態遷移図，状態遷移ログ，変数値表示が同一ステップで同期して更新されることにより，開発者は複数の情報を画面上で往復しながら照合できる。特に変数ウォッチ機能で

は、変数更新ログを逐次反映したスナップショットを各ステップに対応付けて表示するため、「その時点で条件分岐に関与している変数値」を確認する際に、過去のログ行を遡る必要がない。この同時更新による提示は、不正動作の原因究明に要する時間を短縮し、状態遷移と内部変数の関係を直感的に把握することに寄与する。

6.1.3 「単体観察」から「同一時刻の並列比較」へ

複数ノードの不具合は、片方のログだけでは説明できない場合が多い。特にセンサネットワーク等の分散システムでは、送信側では正しく動作しているが、受信側で問題が生じているなど同期タイムラインにより、同一時刻における両ノードの状態・変数を並列に参照できるため、送受信の因果関係や待機の重なりを「対応」として把握できる。第5章の観察例においても、ある時刻における送信側と受信側の状態が同時に提示されることで、相互作用の理解が容易になることを確認した。

複数ノードから構成されるシステムの不具合は、片方のログのみでは説明できない場合が多い。特にセンサネットワークなどの分散システムでは、送信側では正常に動作しているように見えても、受信側で問題が生じているといったケースが存在する。本方式の同期タイムラインでは、同一時刻における複数ノードの状態および変数を並列に参照できるため、送受信の因果関係や待機状態の重なりを「対応関係」として把握することが可能となる。第5章で示した観察例においても、ある時刻における送信側と受信側の状態が同時に提示されることで、ノード間の相互作用を理解しやすくなることを確認した。

6.2 システムの限界・制約

本研究で実装したデバッグ支援システムは、状態遷移ログおよび変数更新ログを解析し、可視化と操作を通じてデバッグ作業を支援する。一方で、適用範囲や入力条件に関していくつかの限界・制約を有する。本節では、実運用を想定した際に留意すべき点を整理する。

6.2.1 同期方式の適用範囲

シリアル受信モードの同期解析は、2台のデバイスから得られるログを共通の時刻軸に

統合し、ノード間の相互作用を把握しやすくすることを目的としている。しかし現状では、同期の基準時刻として、同一PC上でログを受信した際の受信時刻を用いている。そのため、同一PCに接続された端末間の比較には有効である一方、センサネットワークのように地理的に離れたノード間での同期解析や、遠隔環境における絶対時刻整合を前提とする用途にはそのまま適用できない。また、受信処理の揺らぎや通信遅延が大きい環境では、同一時刻とみなす対応付けに誤差が含まれる可能性がある。

6.2.2 ログ形式への依存

本システムは、状態遷移をベースにした組込みシステムにフォーカスしており、状態遷移ログおよび変数更新ログが所定の形式で出力されることを前提として解析を行う。このため、ログ出力形式が異なる既存システムや、出力規約が統一されていない環境に対しては、そのまま適用することが難しく、変換処理や出力の統一が必要となる。また、状態ログの欠落や出力粒度の不統一が生じた場合には、ステップ単位での再構成が困難となり、状態遷移図との対応付けやタイムライン上の解釈が不正確になる恐れがある。

6.2.3 変数追跡対象の限定

変数ウォッチ機能で追跡可能な変数は、ユーザが `PRINT_VAR_LOG()` マクロを用いて明示的に出力したものに限られる。したがって、出力されていない変数については、後から値を遡って確認することはできない。この制約の下では、デバッグ対象の不具合に応じて、監視すべき変数を事前に適切に選定し、必要な変数ログを組込み機器側で出力する運用が求められる。

6.2.4 評価範囲および拡張性に関する制約

本研究では、主として機能成立の確認と、理解容易性に関する定性的観察を中心に評価を行った。そのため、デバッグ時間の短縮量や原因特定精度の向上、工数削減といった効果を定量的に示す比較評価は今後の課題である。また、ノード数の増加に伴うブラウザ上への表示の複雑化や、ログ量増大時の探索性・描画負荷については追加検討が必要である。

6.3 今後の課題

本システムの実運用に向け、今後取り組む課題を以下に整理する。

6.3.1 遠距離環境での同期解析への対応

現状の同期タイムラインは同一PCで受信した時刻を基準としているため、遠距離に分散したノード間では利用できない。今後は、各ノードがログに時刻情報を付与し、ノード間の時刻ずれを補正して統合できる仕組みを検討する。

具体的には、(i) デバイス時刻とPC時刻のオフセット推定、(ii) 送受信イベントの対応関係を用いた相対同期、(iii) GPS や 1PPS 等の外部同期源を利用した絶対同期、などが候補となる。これにより、センサネットワークのように地理的に離れたノード間でも、同期解析の適用範囲を拡大できると考える。

6.3.2 ログ形式依存の低減（パースの強化）

本システムは所定形式のログを前提としているため、形式が異なる環境では解析が困難である。今後は、ログ形式の変換処理を用意するとともに、表記ゆれを許容して状態・変数を抽出できるパース手法を整備する。

6.3.3 AIを用いたログパースの補助

ログ形式のゆれや混在メッセージに対して解析を安定させるため、AIの利用を検討する。具体的には、生ログ各行を「状態遷移」「変数更新」「その他」に分類し、状態名や変数名・値を抽出して、states/vars 生成に利用する。ただし推定結果は候補として扱い、ユーザが確認できる形で提示するのが望ましい。

6.3.4 変数追跡の運用負担の軽減

変数追跡は `PRINT_VAR_LOG()` によりログ出力した変数に限られるため、監視対象の選定漏れが起こり得る。今後は、監視すべき変数のテンプレート化や候補提示などにより、必要な変数ログを出力しやすくする工夫を行う。

6.3.5 効果の定量評価

本研究は機能成立の確認と定性的観察が中心であり、効果を数値で示せていない。今後は、従来のコンソール追跡と比較し、原因特定までの時間、手戻り回数、原因特定の正確性などの指標で評価する。

6.3.6 多ノード化・大規模ログへの対応

現状はファイル解析モードでは1ノード、同期解析では2ノードを前提にした設計であり、多ノード化やログ量増大に対して探索性が課題となる。今後は、フィルタや検索、重要イベントへのジャンプ等を追加し、多ノード・大規模ログでも目的箇所へ到達しやすくする。

6.4 本研究のまとめ

本研究では、無線センサネットワークのプロトコル検証におけるデバッグ作業を支援するため、ブラウザ上で動作するデバッグ支援システムを設計・実装した。本システムは、状態遷移ログおよび変数更新ログを解析し、状態遷移の可視化、変数値追跡、ステップ実行による逐次確認を可能にする。さらに、複数ノードのログを統合して同期タイムラインを生成し、同一時刻における状態・変数の比較を通じて、相互作用やタイミング関係の把握を支援した。

評価では、ファイル解析モードとシリアル受信モードにおいて、ログから `states/vars` を生成し、可視化・操作が一貫して行えることを確認した。また、同期タイムラインにより、送受信関係や待機の重なり、衝突・再送といった現象を複数ノードの対応として読み取りやすくなる点を定性的に示した。一方で、本評価は機能成立の確認を主目的としたため、デバッグ時間の短縮や原因特定精度の向上といった効果を定量的に示すには至っていない。

今後は、効果を測る評価指標の設計と比較実験を行うとともに、入力ログの信頼性向上、検索・注釈などの探索支援、多ノード化に対応したUI設計を進める。さらに、同期機構を拡張し、遠隔環境でも時刻整合を保証できるログ統合を実現することで、本システムの適用範囲を広げる予定である。

参考文献

- [1] IoT Analytics: "State of IoT 2024: Number of Connected IoT Devices Grows 16% to 16.6 Billion", 2024.
- [2] Grand View Research: "Industrial Wireless Sensor Network Market Size, Share & Trends Analysis Report, 2024–2030", 2024.
- [3] 旭 健汰, アサノ デービッド, 不破 泰: "無線モデムを用いたセンサネットワーク MAC プロトコル検証システムの開発", 信学技報, NS2023-147, pp.120–125, Dec. 2023.
- [4] 小林 遼, アサノ デービッド, 不破 泰: "センサーネットワーク検証システムにおける遷移図を用いた MAC プロトコル設計環境の開発", 信学技報, NS2023-148, pp.126–131, Dec. 2023.
- [5] 小林 侑生, アサノ デービッド, 不破 泰:
- [6] PlatformIO Labs, "What is PlatformIO?", <https://docs.platformio.org/en/latest/what-is-platformio.html>, 参照 Oct. 15, 2025.
- [7] Mermaid.js, "Overview," <https://mermaid.js.org/intro/>, 参照 Oct. 15, 2025.
- [8] Mermaid.js, "State Diagram Syntax (stateDiagram-v2)," <https://mermaid.js.org/syntax/stateDiagram.html>, 参照 Oct. 15, 2025.
- [9] Eclipse Foundation, "MQTT: Message Queuing Telemetry Transport," <https://mqtt.org/>, 参照 Oct. 15, 2025.
- [10] 園田 継一郎, 不破 泰, アサノ デービッド, "無線センサーネットワークの端末・中継機における送信タイミング決定時間短縮方法の検討," 信学技報, NS2022-138, Dec. 2022.